

Break the Login – Atacarea și securizarea autentificării

Proiect Dezvoltarea Aplicațiilor Software Securizate | an universitar
2025 – 2026 | realizat de Năvodaru Denisa

GitHub repo link : [here](#)

Demo video drive link: [here](#)

1. Introducere

Aplicația „Break the Login” se axează pe aspectele de login/register ale unei aplicații web interne folosită de compania fictivă „AuthX”, ai căror angajați au semnalat probleme de securitate. În prima parte este aplicația vulnerabilă (pe branch-ul „vulnerable”) în care nu sunt respectate normele de securitate, ulterior aducând modificări (pe branch-ul „secure”), care reduce considerabil eficiența potențialelor atacuri asupra acesteia.

Ca funcționalități, un angajat își poate crea cont prin pagina de Register, după care se poate loga pe pagina de Login (dacă este confirmată crearea contului). În cazul în care își uită parola poate fi redirecționat spre o pagină de resetare. În poziția de analyst acesta poate crea tichete și a le vizualiza pe cele proprii. Ca manager, poate vizualiza toate tichetele create de orice angajat, să schimbe statusul (open/în progress/resolved) și să vadă audit logs în care poate vedea acțiunile din cadrul aplicației și cine le-a efectuat când.

Este o aplicație web construită cu Flask cu o arhitectură stratificată simplă. Fișierul principal (app.py) se ocupă atât de partea de controller, cât și logica de business, în el gestionând rutele, autentificarea, autorizarea și funcționalitățile. Interfața e destul de simplă și redată prin template-uri HTML. Starea utilizatorilor este reținută prin sesiuni. În database.py este gestionată conexiunea la o bază de date SQLite și partea de audit logs. Schema cu cele 3 entități (users, tickets, audit_logs) este în schema.sql. Pentru că am folosit CSP, script-ul și stylesheet-ul pentru templates sunt în folderul static.

2. Set-up mediu

Ca mediu virtual am folosit Oracle VirtualBox pe care am instalat Windows 10 (64-bit) și m-am asigurat că sunt conectată pe aceeași rețea cu host-ul cu conexiune Bridged Adapter.

Pentru baza de date am folosit SQLite 3 deoarece am mai lucrat cu el înainte și a fost ușor de navigat. Ca framework am ales Python Flask, fiind destul de popular și cu multe resurse utile și accesibile pe internet.

Pentru a avea acces la aplicație în VM am adăugat în instrucțiunea de run `host='0.0.0.0'` și `port = 5000` și am rulat direct din terminal „python app.py” pentru a trece prin funcție. În urma rulării cu aceste setări, aplicația pe VM devine accesibilă pe adresa returnată în terminal.

```
(.venv) PS D:\00\OneDrive\faculta\DASS\Proiect_DASS> python app.py
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://192.168.1.166:5000
Press CTRL+C to quit
192.168.1.165 - - [29/Apr/2026 11:39:37] "GET /register HTTP/1.1" 200 -
192.168.1.165 - - [29/Apr/2026 11:39:37] "GET /static/script.js HTTP/1.1" 200 -
192.168.1.165 - - [29/Apr/2026 11:39:37] "GET /static/styles.css HTTP/1.1" 200 -
```



3. Implementare MVP

3.1. Varianta vulnerabilă

Pe partea de register, user-ul completează formularul cu un email (trebuie să fie unic în baza de date), o parolă oarecare și să își aleagă unul dintre rolurile disponibile (analyst/manager). Dacă completează toate câmpurile și adresa de email este unică, atunci va fi afișat un mesaj că se poate loga. Parola este reținută în clar (pw_hash reține password direct fără vreo funcție de hash) și la login este comparată cu cea din formular, la fel ca email-ul. Dacă se potrivesc una cu alta, utilizatorul va fi redirecționat către profilul său. Dacă parola este uitată, poate fi selectat butonul de „Forgot password” care redirecționează către o pagina unde va fi introdus email-ul și va primi un token (1234, ușor de ghicit) pentru resetarea parolei. Dacă token-ul și parolele introduse (new + confirm) se potrivesc, parola va fi modificată. Deoarece managerul are acces la mai multe resurse decât un analyst, fac o verificare a rolului pentru a ști ce trebuie afișat pe profilul utilizatorului. Sesiunea unui user durează foarte multe zile by default.

Orice utilizator poate crea tichete de pe profilul său printr-un formular simplu. Ulterior, un analyst își poate modifica titlul, descrierea și severitatea propriilor tichete, dar statusul poate fi modificat doar de manageri (care au acces la tot ce ține de tichete, în esență).

Audit logs sunt create după fiecare acțiune relevantă și nu pot fi modificate de către utilizatori. Acestea rețin cine a efectuat ce acțiune, asupra cărei entități și în ce moment. Managerii le pot vizualiza din propriul profil într-un tabel.

Pentru Search la tichete am folosit un query parametrizat.



Captura de ecran este de pe branch-ul secure, însă bucățile de cod prezente sunt la fel în ambele variante (vulnerabilă + securizată).

3.2. Varianta securizată

În urma modificărilor aduse, formularul de register devine puțin mai riguros, cerând o complexitate a parolei de minim 8 caractere, cel puțin un simbol, o cifră, un caracter mic și unul mare. Dacă email-ul este unic, parolele (trebuie introdusă de 2 ori pentru confirmare) și rolul sunt completate corect, utilizatorul va vedea un mesaj că se poate loga. O dată logat cu credențialele corecte, acesta va avea acces la profil cu o sesiune de 10 minute prin token-uri CSRF. Dacă se duce pe Forgot password, token-ul nu va mai fi la fel de ușor de spart ca înainte (chiar va fi foarte dificil). Criteriile pentru modificat parola rămân aceleași, dar token-ul generat în această variantă este mult mai sigur. De asemenea, la 5 încercări greșite de login (parole greșite pentru un mail existent), utilizatorul va avea statusul contului „locked”. Într-o aplicație reală, acesta ar trebui să apeleze la Customer Support pentru a-i fi reactivat contul cu un token nou de resetare a parolei, dar în aplicația locală nu permit deblocarea contului. Între această decizie și a pune un timer de o ora până la deblocare, m-am gândit că un utilizator poate fi targetat de un individ/organizație care ar putea fi destul de dedicat încât să încerce câte 5 parole pe oră până reușește să intre în cont, așa că am vrut să elimin acest scenariu.

Partea de CRUD și Search sunt la fel ca la varianta vulnerabilă. Am făcut Search-ul fără SQL injection deoarece aveam deja această vulnerabilitate prezentă.

4. Prezentare vulnerabilități

4.1. Password policy slab (OWASP – Testing for Weak Password Policy – ID: [WSTG-ATHN-07](#))

Parolele slabe reprezintă un risc uriaș în securitate, fiind ușor de ghicit de persoane oarecare. De multe ori este nevoie de informație minimală despre persoana respectivă pentru a avea indicii legate de parola acestora. În cazul aplicației mele, în prima parte parola unui user era chiar numele din email (ex: user@gmail.com cu parola user). Nu impun nicio restricție despre complexitatea parolei, aceștia fiind liberi să își pună parole foarte des ghicite (1234, qwerty etc.). Acestea reprezintă un risc mare pentru utilizatori și în afara aplicației curente, un atacator având posibilitatea de a reutiliza credențialele ghicite și pe alte site-uri pe care suspectează că utilizatorul le-ar frecventa cu un cont personal.

4.2. Stocare nesigura a parolelor (OWASP – Plaintext Password Storage – [CWE-256](#))

Prin stocarea parolelor în baza de date fără nicio funcție de hash, acestea devin ușor accesibile de către atacatori prin config files unde sunt stocate proprietățile contului prin script-uri speciale. În prima versiune a aplicației mele, parolele sunt stocate în clar.

4.3. Brute force / lipsa rate limiting (OWASP – Testing for Weak Lockout Mechanism – ID: [WSTG-ATHN-03](#))

Prin lipsa unui mecanism de a limita încercările de login într-un cont există riscul ca un atacator să folosească tool-uri diverse pentru a încerca nenumărate parole la un cont existent, până când acesta va reuși să se logheze. De asemenea, este important ca un utilizator să nu își piardă accesul la cont prea ușor. Este recomandat ca după 3-5 încercări contul să fie flagged într-un fel sau altul pentru a preveni alte încercări de login greșite din partea unui potențial atacator. În versiunea mea vulnerabilă nu există niciun fel de sesizare a acestor tentative de login eșuate.

4.4. User enumeration (OWASP – Testing for Account Enumeration and Guessable User Account – ID: [WSTG-IDNT-04](#))

Într-o aplicație nesecurizată, un atacator poate pune la un loc o listă de utilizatori existenți prin intermediul formularului de Login care anunță persoana ce completează care dintre credențiale este greșit. O aplicație ar trebui să înștiințeze un utilizator de existența unei erori fără a compromite securitatea datelor prin output-uri diferite la erori diferite. În prima versiune, utilizatorul este înștiințat dacă user-ul este greșit sau parola.

4.5. Gestionare nesigură a sesiunilor (OWASP – Improper Session Handling – [link](#))

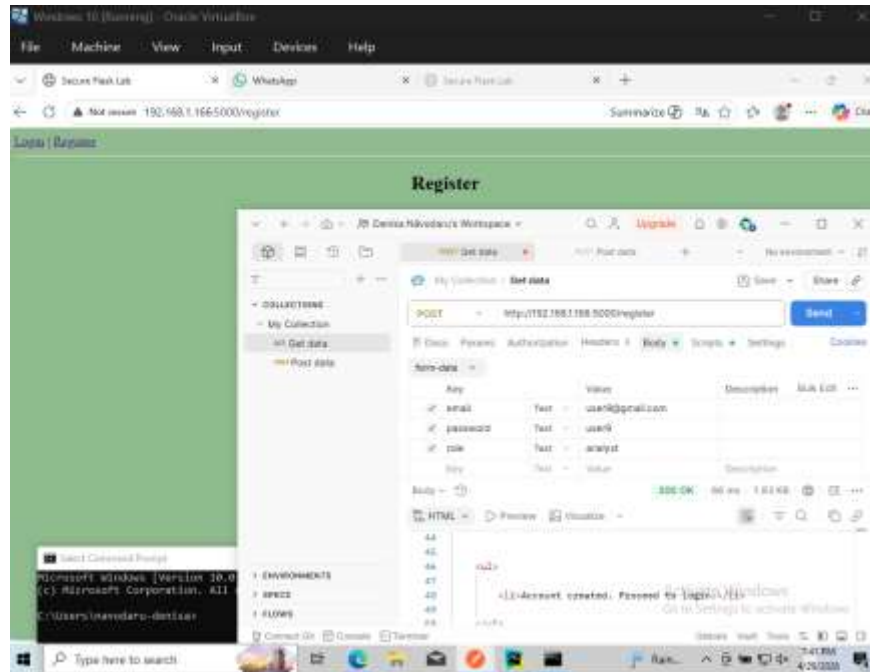
Un atacator poate avea acces la token-urile sesiunii pentru a păcăli aplicația că este utilizatorul respectiv pentru a avea acces la informații/procese importante. By default, folosesc o sesiune cu time-out foarte lung și nu gestionez cookies într-un mod sigur.

4.6. Resetare parola nesigură (OWASP – Testing for Weak Password Change or Reset Functionalities – ID: [WSTG-ATHN-09](#))

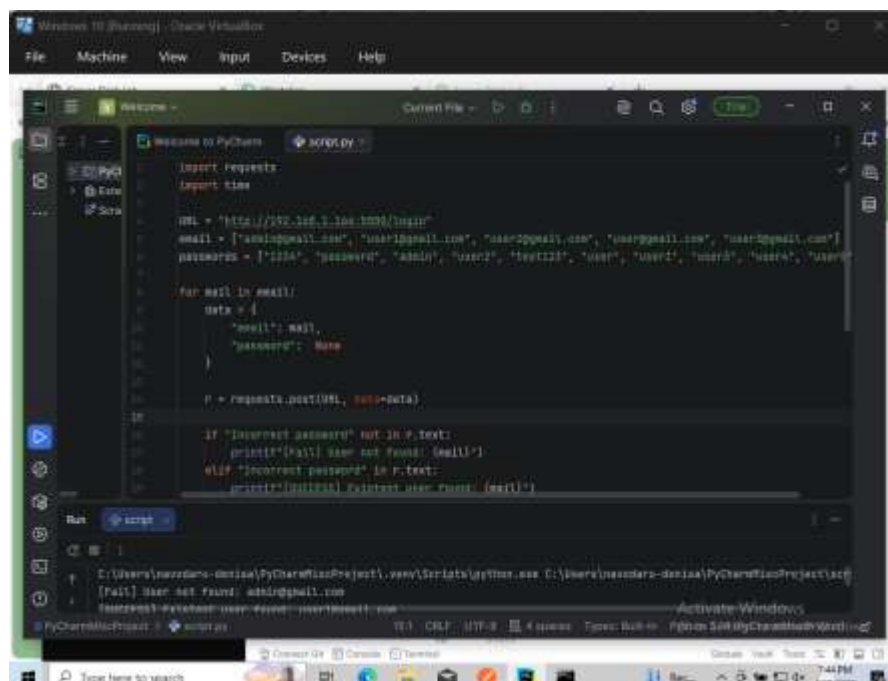
Utilizatorii au nevoie de o plasă de siguranță în caz că își uită parola de la contul personal, astfel încât să își recapete accesul la el. Acest lucru poate fi mai sigur prin contactarea unui Customer Support, dar în viața reală este mult mai util să existe un formular sau ceva asemănător la îndemâna oricărui utilizator. Multe aplicații folosesc serviciul de email pentru a trimite un OTP sau orice fel de single use link către o pagină de resetare a parolei. Într-o versiune mai veche a aplicației mele vulnerabile puneam la dispoziție un formular pentru email și parola nouă în care oricine putea modifica parola unui cont existent. Deși este o vulnerabilitate mare, am modificat cu un token de reset pentru a arăta că și acesta poate fi vulnerabil dacă nu este utilizat corespunzător. Token-ul de resetare este mereu 1234, deci foarte ușor de ghicit de un atacator care poate modifica orice parolă a conturilor a căror existență a verificat-o.

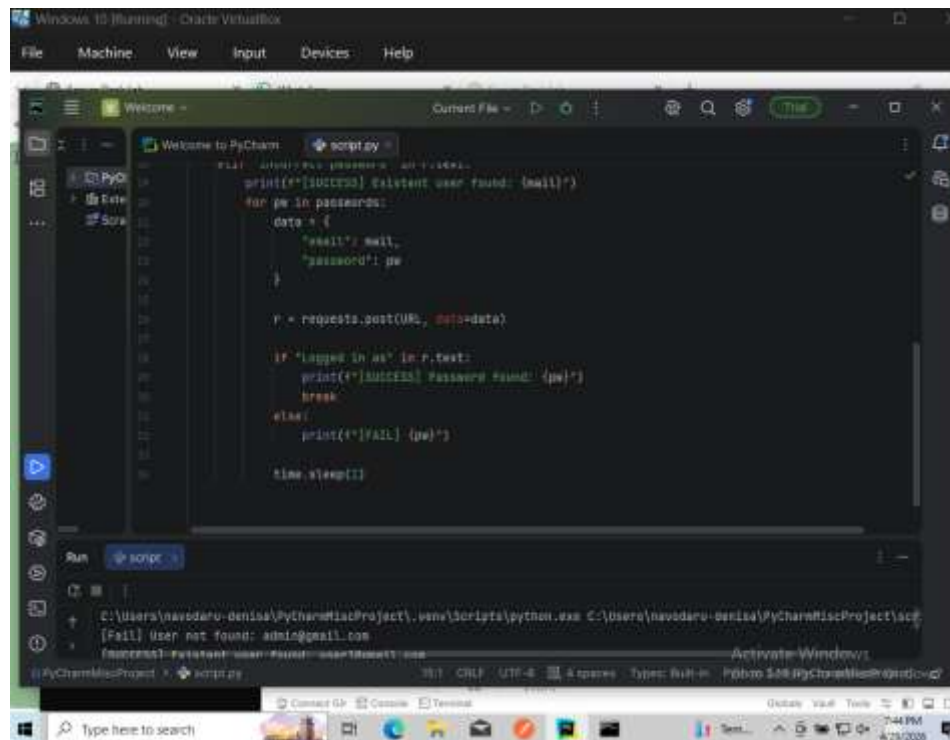
5. Demonstrare atac

5.1. Password policy slab



După cum se vede, am avut voie sa creez un cont cu parola slabă „user9”. Așadar, cu un script care verifică parole pentru un email existent se va putea afla relativ ușor parola. Am folosit următorul script ca demo:





5.2. Stocare nesigură a parolelor

Project_DASS - vulnerable

Terminal Local (2) Local

Install the latest PowerShell for new features and improvements! <https://aka.ms/PSWindows>

[.view] PS D:\SD\src\drive\feoulta\BASS\Project_DASS> python3 app.py

SQLite version 3.33.0 2026-04-09 11:41:38

Enter ".help" for usage hints.

sqlite> select * from users;

id	email	password_hash	role	created_at	locked	failed_attempts	reset_token	reset_token_expiry
1	user@gmail.com	script:32768:8:184c10PcSr9pr10f2P5-c1e983182d0d44697b8427b0b51a20adba7eb76478ec79201e93310e54700cc54cedc4d0f26b0ee05d6025466ae36fd0f7242b7e9f420f960c9c1784ea48c3	analyst	2026-04-24 14:06:35	0	0	AgYFGR_jd08Pz-5CzJ3a0ILqWq-hip02Lc95c150c	2026-04-24 15:54:20.243141+00:00
2	user1@gmail.com	script:32768:8:184c10PcSr9pr10f2P5-8f53026e5a785a386ce2ca971e280233ba5997a3a324619642335386b3e4f53168324286897960ea36f415c8b2f7c707cfa5318a9c015426a9a619701ee15	manager	2026-04-24 14:06:40	0	0	NULL	NULL
3	user2@gmail.com	user2	analyst	2026-04-24 14:07:49	0	0	NULL	NULL
4	user3@gmail.com	user3	analyst	2026-04-24 14:55:19	0	2	NULL	NULL
5	user4@gmail.com	user4	manager	2026-04-24 16:55:47	0	0	NULL	NULL
6	user5@gmail.com	user5	manager	2026-04-25 20:28:33	0	0	NULL	NULL

Project_DASS - app.py

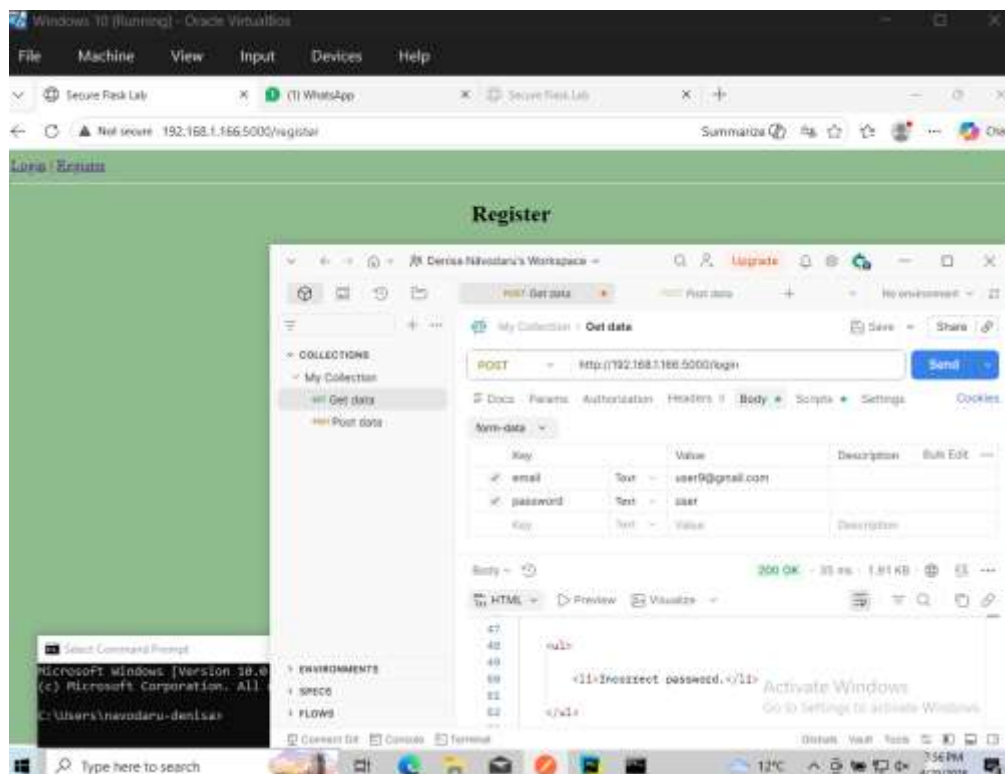
40:45 CRLF UTF-8 4 spaces Python 3.10 (Project_DASS)

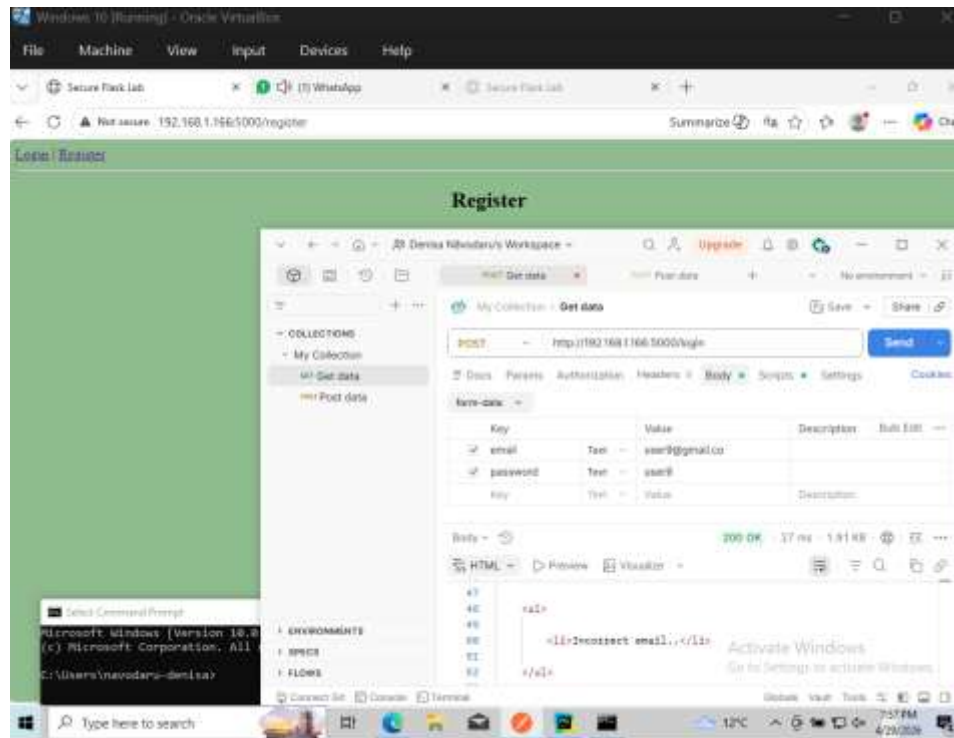
Presupunând că sunt un atacator cu acces la baza de date, este suficient să îmi selectez utilizatorii ca să văd ce e stocat în parolele lor. În captura de ecran am atât parole hash-uite, cât și salvate în clar, deoarece unele conturi au fost create pe varianta vulnerabilă, iar altele pe cea securizată, sau am modificat parola creată pe o versiune de pe cealaltă versiune.

5.3. Brute force / lipsă rate limiting

Cum am arătat la 5.1 și urmează și la 5.4, pot folosi acel script pentru a încerca nenumărate parole până când voi ajunge la cea asociată email-ului ales. Nu se întâmplă nimic după vreun număr de încercări, orice atacator e liber să încerce un miliard de parole greșite până la cea norocoasă.

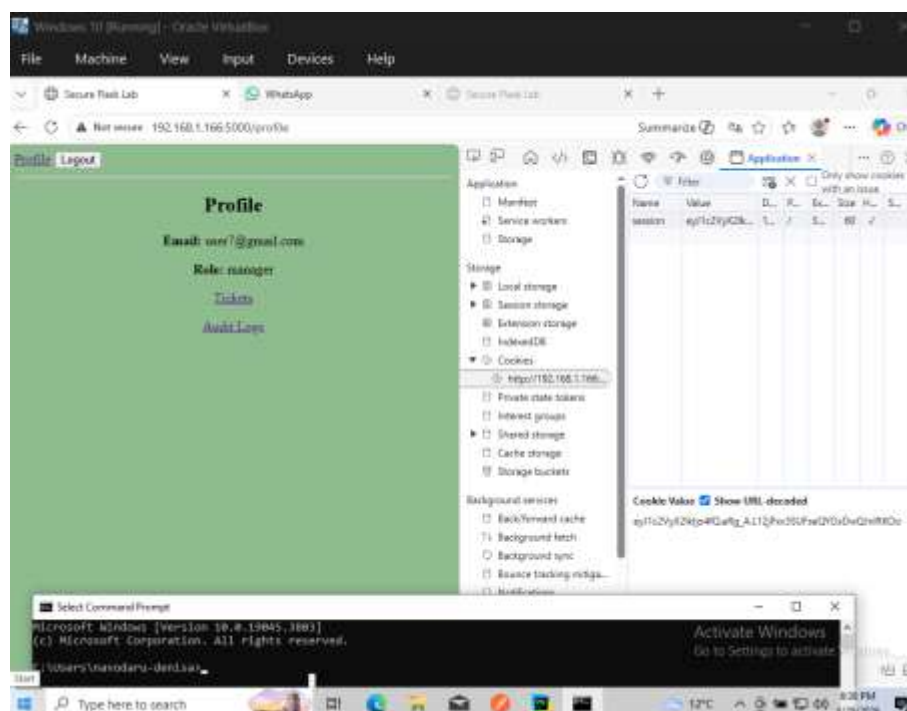
5.4. User enumeration

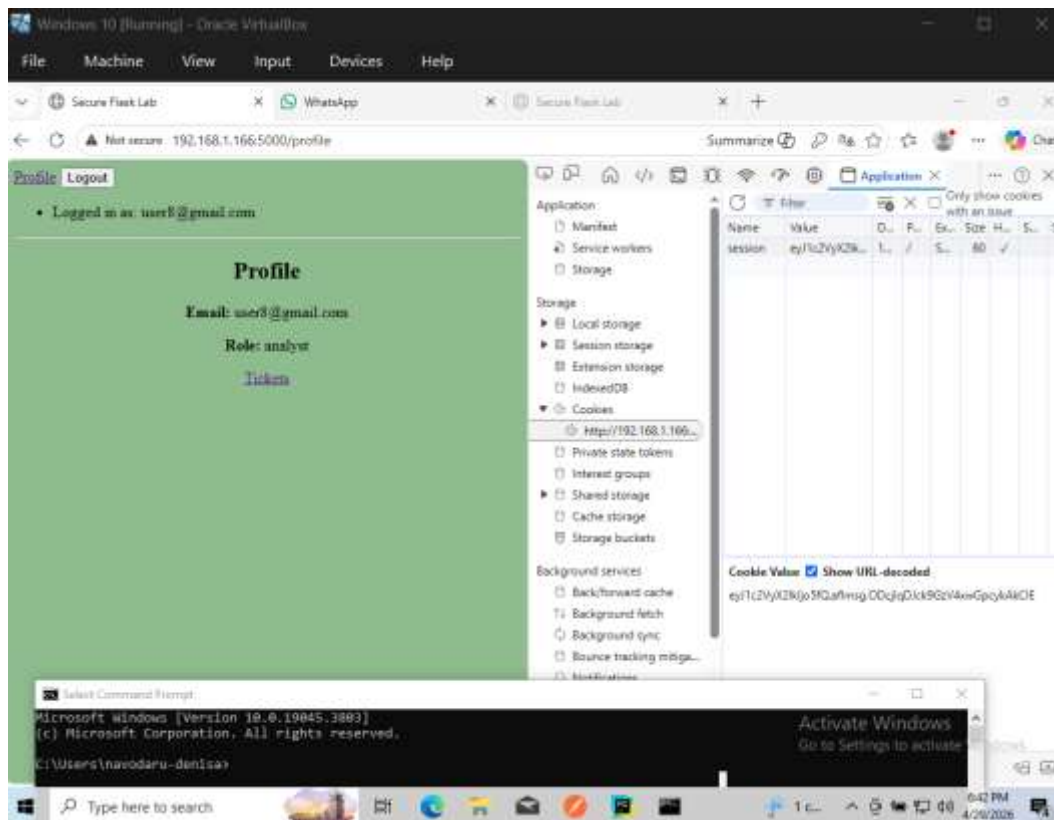




Așa cum se poate observa, în funcție de care credențial este greșit voi primi un avertisment diferit, cea ce permite aceluiași script de mai sus să găsească utilizatorii existenți și să verifice parole diverse pe un mail confirmat ca fiind în baza de date.

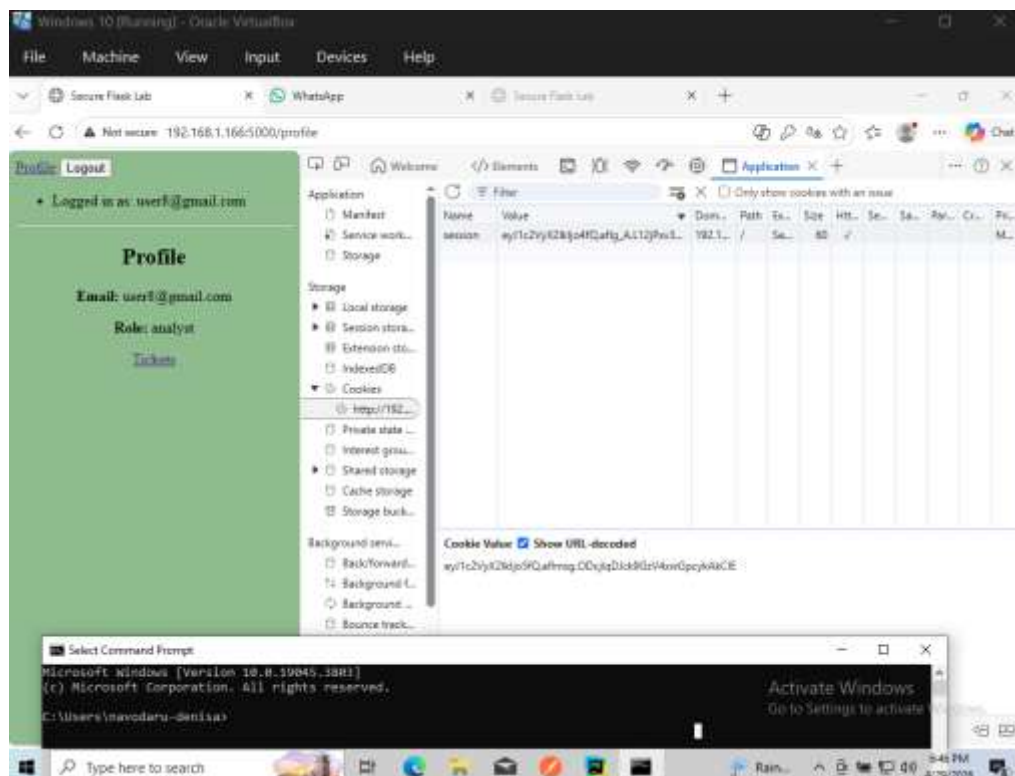
5.5. Gestionare nesigură a sesiunilor



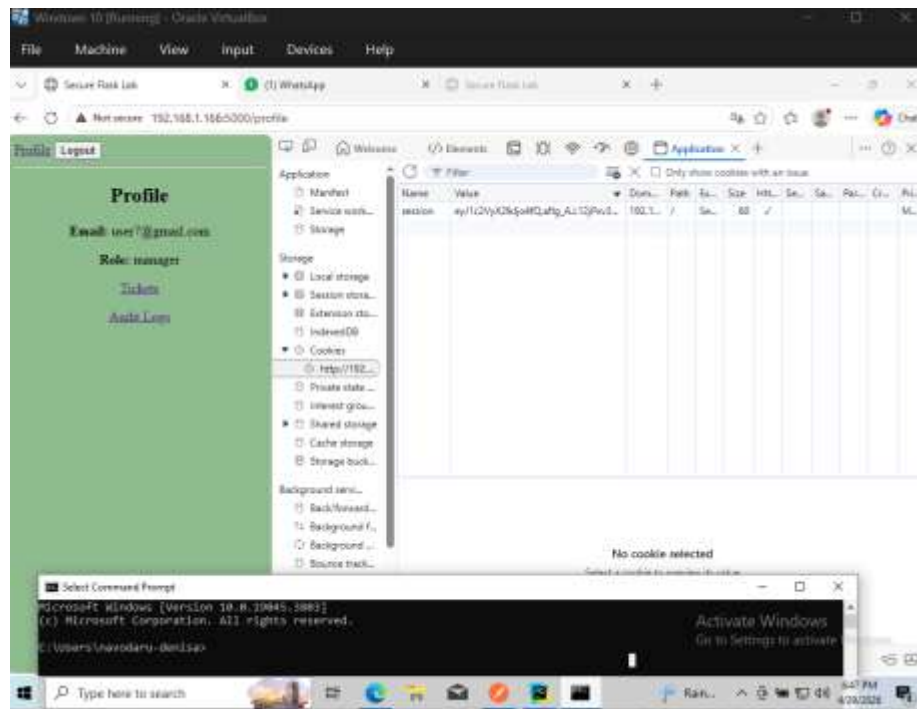


User7 cookie value: eyJ1c2VyX2lkIjo4fQ.afIg_A.L12jPxx3SUFseQYOxDwQJnlRKOo

User8 cookie value: eyJ1c2VyX2lkIjo5fQ.afImsg.ODcjIqDJck9GzV4xwGpcykAkCIE

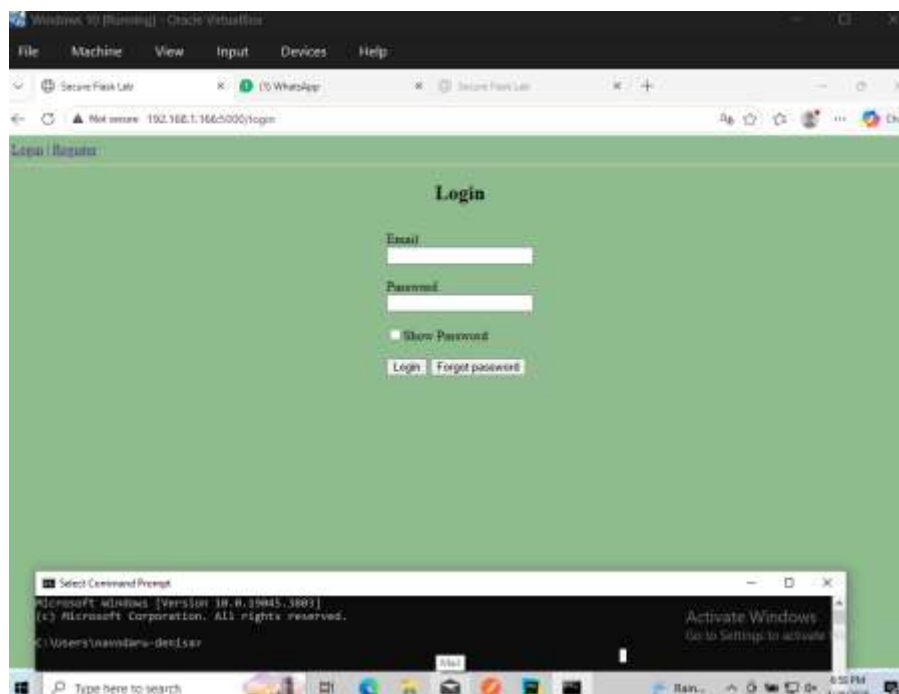


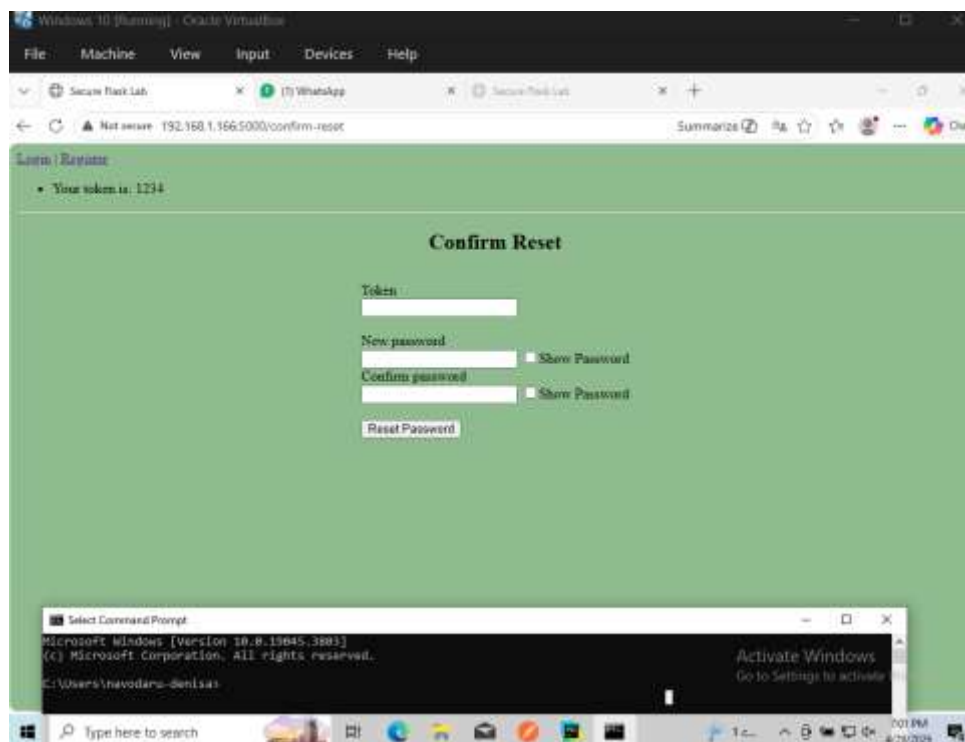
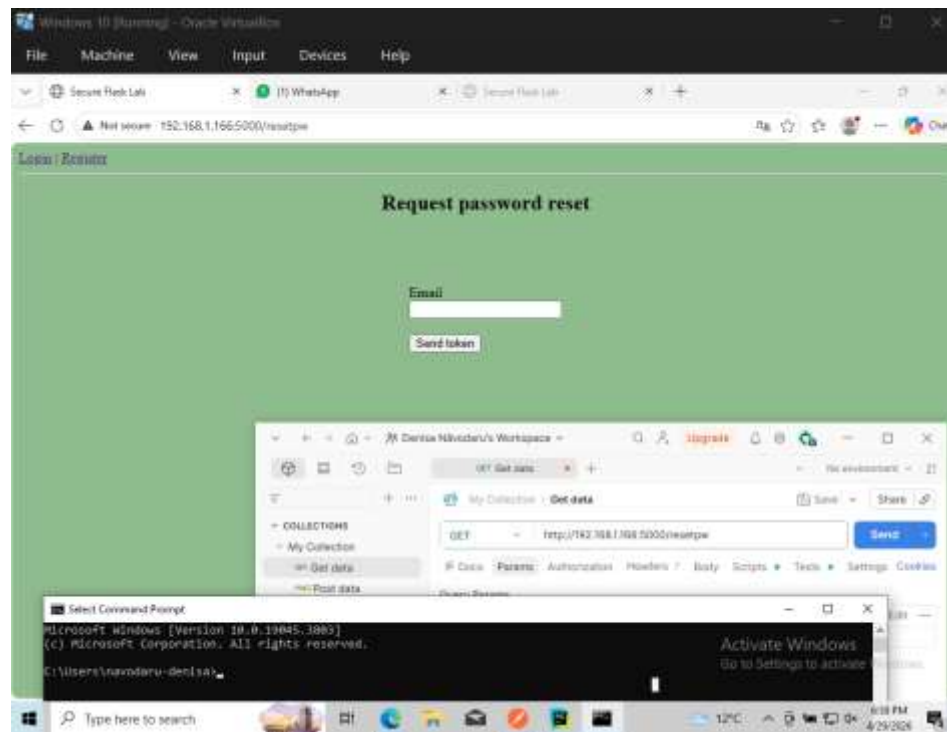
Am introdus valoarea de la User7 de pe profilul lui User8. Rezultatul în urma unui refresh:



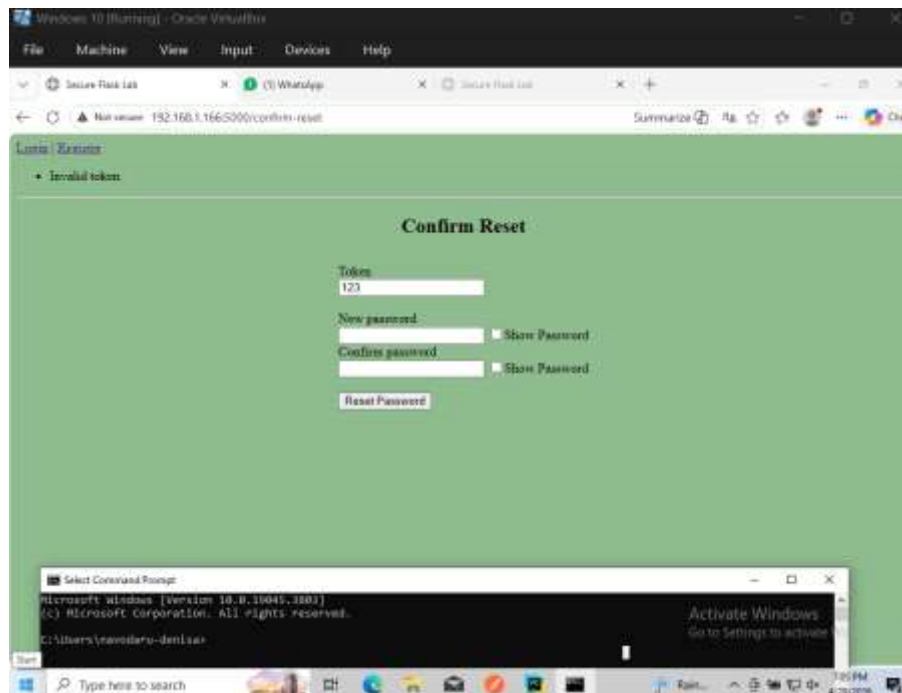
După cum se poate observa, prin modificarea valorii am reușit să intru în sesiunea unui alt user.

5.6. Resetare parola nesigură

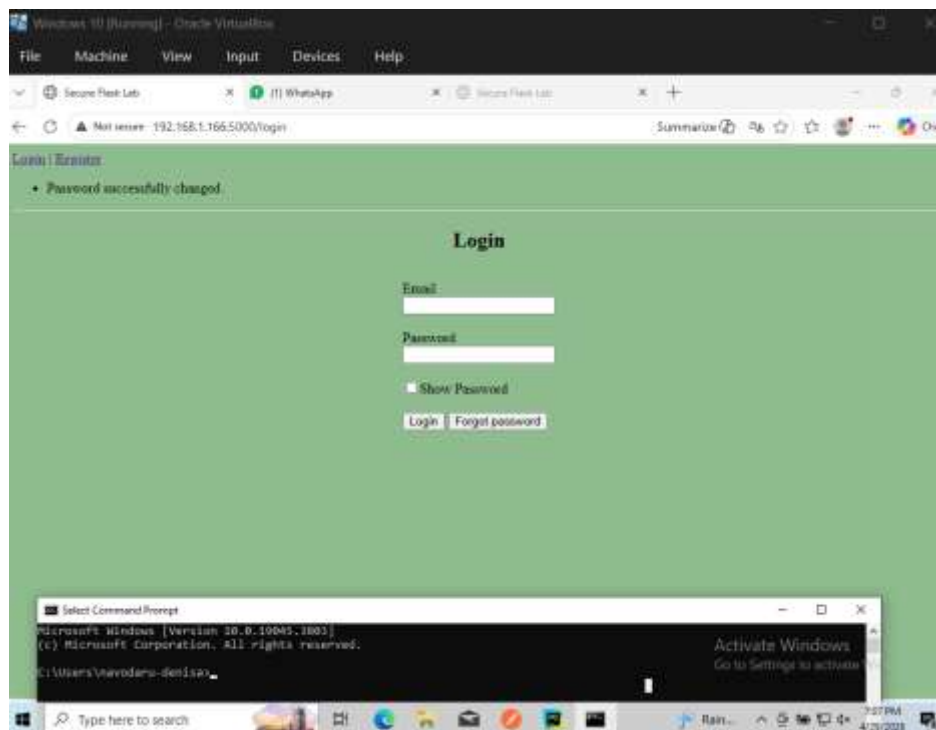




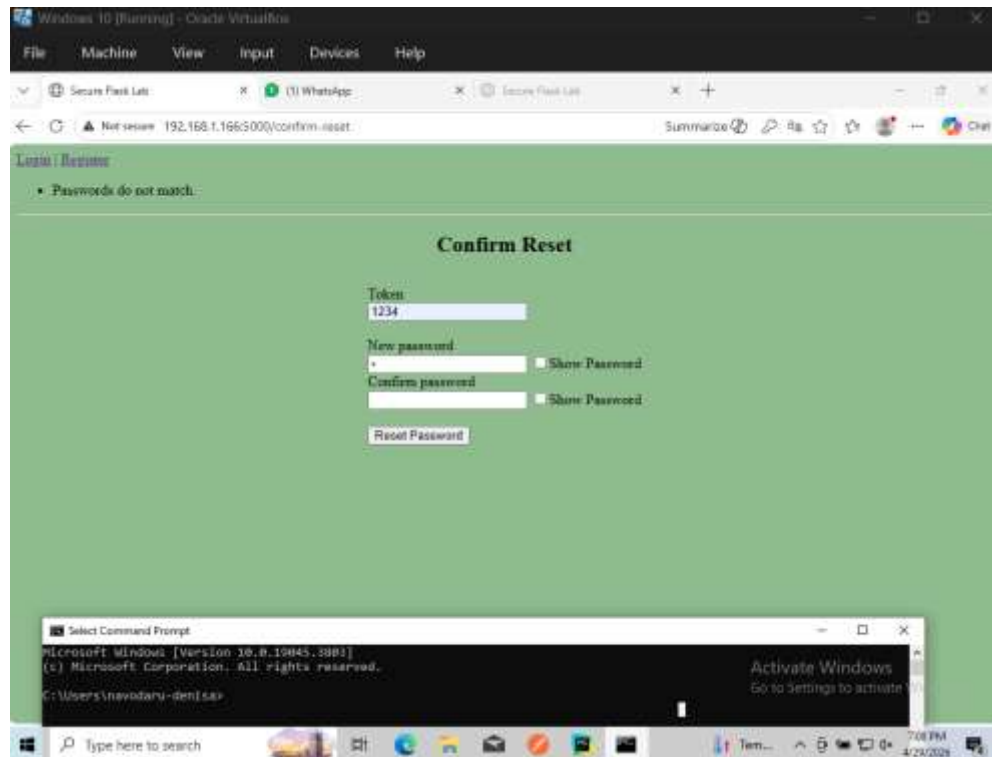
În aplicația mea am dat flash cu token-ul, însă într-o aplicație reală ar trebui folosit un single use link primit pe email sau ceva similar, dar care ar avea la bază tot un token. Aici problema este că token-ul este mereu 1234, deci cu câteva încercări orice atacator poate profita de el pentru a reseta parola la orice cont pe care îl știe ca fiind existent. De altfel, dacă introduc un token greșit voi primi o înștiințare.



Dar dacă introduc token-ul corect și las câmpurile de parolă goale:



Parola a fost modificată pentru că nu impun criterii de complexitate asupra ei, deci parola goală este acceptată. Presupunând totuși că nu ar fi acceptată o parolă goală, cu un token corect și un mismatch între parole:



Aşa atacatorul poate verifica în funcție de avertismente dacă a ghicit sau nu token-ul de reset, deci este doar vorba de timp și resurse până la un atac.

6. Analiză impact

6.1. Password policy slab

Script-ul folosit de mine nu este unul tipic de atacator, aceștia ar folosi un generator de email-uri și parole pentru a le testa adecvat, însă prin cunoștințele și resursele mele limitate acest script tot reușește să arate vulnerabilitatea. Parolele slabe vor fi întotdeauna primele sparte, fiind necesar un script mult mai simplu și niște generări mai rapide pentru a le găsi și asocia cu un cont.

6.2. Stocare nesigură a parolelor

Cu parolele stocate în clar este doar o problemă de acces la baza de date. Desigur, nu e un pas ușor, însă în situația în care un atacator reușește să aibă acces la aceasta, parolele vor fi total compromise fără o funcție de hash. Ar fi compromise și cu un hash slab fără salt, deoarece ar fi ușor de descifrat cu câteva parole la dispoziție.

6.3. Brute force / lipsă rate limiting

Deși e important ca un utilizator să aibă acces la contul său după ce își greșește de câteva ori parola, este și mai important să știm când trece de la un „am uitat parola, dar sigur e una din astea 3 pe care le folosesc” la un „nu știu parola, vreau să intru în contul acesta”. Lipsa de contorizare a tentativelor de login facilitează script-uri precum cel folosit de mine prin nenumăratele încercări care îi stau atacatorului la dispoziție.

6.4. User enumeration

Script-ul meu are un dicționar limitat de utilizatori și parole, așa cum am menționat anterior, însă principiul este cel relevant. În funcție de mesajul afișat după o încercare de login se poate trece la încercarea unui email nou sau la parole pentru un utilizator existent. Asta este o veste foarte bună pentru atacatori care primesc informații ce facilitează un atac mai rapid. După ce script-ul verifică login-ul pe mai multe conturi, aceștia vor avea o colecție de conturi la care au acces.

6.5. Gestionare nesigură a sesiunilor

Deși e o valoare lungă, prima parte a cookie-ului este la fel (eyJ1c2VyX2lkIjo4fQ.) și un script realizat de un atacator poate profita de această vulnerabilitate pentru a intra într-un cont existent. Atacuri de tip XSS sunt atacuri specifice în care atacatorii injectează script-uri malițioase în aplicații, care se execută în browser-ul utilizatorului și permite atacatorilor să fure cookies din sesiune. În implementarea vulnerabilă, aplicația este foarte vulnerabilă la aceste atacuri.

6.6. Resetare parolă nesigură

Din cauza token-ului care este mereu același și care este, în plus, foarte previzibil, un atacator are la dispoziție posibilitatea de a rula un script pentru a ghici token-ul până când avertismentul nu mai e „Invalid token” ci „Passwords do not match”. Din acel punct poate folosi

token-ul pentru orice cont dorește să spargă. Este clar foarte periculos pentru că pe lângă faptul că se pot face acțiuni compromițătoare din contul utilizatorului, acesta va fi blocat din propriul său cont din cauza unei noi parole pe care nu o cunoaște.

7. Implementare fix

7.1. Password policy slab

[illegible]

The screenshot displays a VS Code editor with a Python Flask application. The file explorer on the left shows the project structure: `confirmsent.html`, `base.html`, `login.html`, `app.py`, `database.py`, `schemas.py`, `logout.html`, `users`, and `utils_logs`. The `app.py` file is open, showing the following code:

```
<register>
app.route('/register', methods=['GET', 'POST'])
def register():
    if request.method == 'POST':
        email = (request.form.get('email') or '').strip()
        password = request.form.get('password') or ''
        password2 = request.form.get('password2') or ''
        role = request.form.get('role') or ''

        if not email or not password or not password2 or not role:
            flash("Email, password and role are required.")
            return render_template('register.html')

        if password != password2:
            flash("Passwords must match.")
            return render_template('register.html')

        pw_check = password_complexity(password)
        if not pw_check['password_ok']:
            flash("Password is too weak. Must have at least 8 characters, one digit, one symbol, one upper and one lower character.")
            return render_template('register.html')

        pw_hash = generate_password_hash(password)

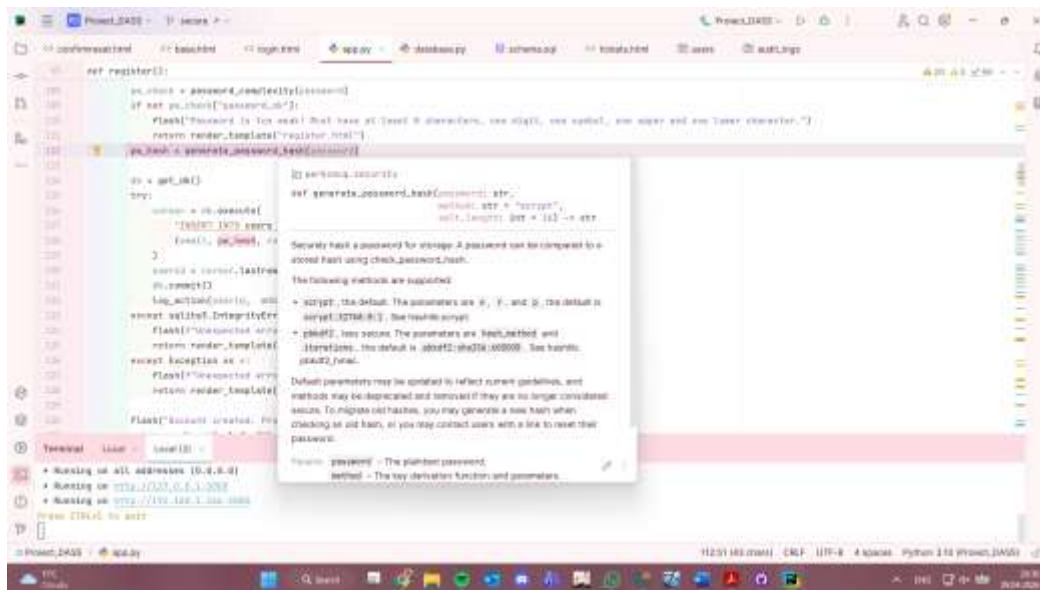
        db = get_db()
        try:
```

The terminal at the bottom shows the application running on `http://0.0.0.0:5000`. The status bar at the bottom indicates the file encoding is UTF-8, the editor is in Python 3.10, and the Python interpreter is set to the project's environment.

Am creat o funcție ajutătoare prin care verific toate criteriile generale ale unei parole sigure (minim 8 caractere, un simbol, o cifră, un caracter mic și unul mare). Am intenționat să afisez o

eroare pentru fiecare criteriu care nu a fost urmat, însă mi s-a părut puțin overkill, în schimb arăt o eroare care specifică ce trebuie să conțină parola. Am importat librăria re pentru a folosi funcția de regex pe parola.

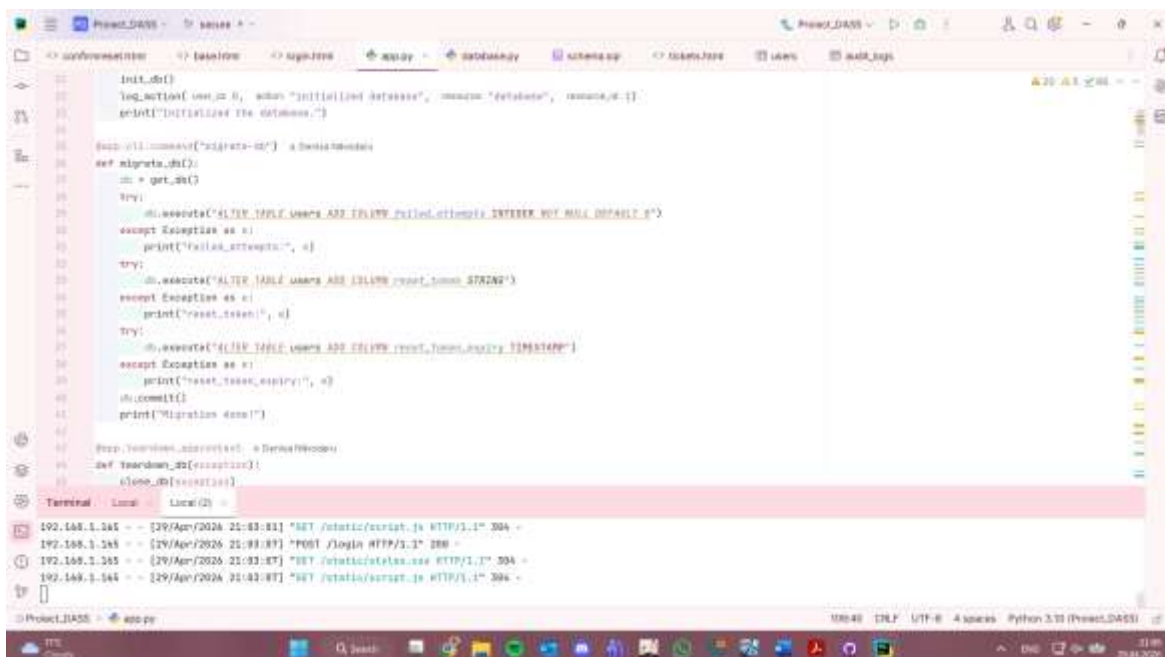
7.2. Stocare nesigură a parolelor



Pentru a stoca parolele într-un mod mai sigur, am folosit funcția `generate_password_hash` din librăria `werkzeug.security`, care folosește în spate funcția `script` cu un salt default de 16.

7.3. Brute force / lipsă rate limiting

Pentru a reduce numărul de încercări de login, am adăugat câteva atribute în tabela de users.



Cu aceste modificări, acum pot contoriza încercările de login ale unui utilizator, iar dacă acesta greșește parola de 5 ori, contul îi va fi blocat.



```

112 return render_template("register.html")
113
114 @app.route("/login", methods=['GET', 'POST'])
115 def login():
116     if request.method == 'POST':
117         email = request.form.get("email") or ""
118         password = request.form.get("password") or ""
119
120         db = get_db()
121
122         user = db.execute(
123             "SELECT * FROM users WHERE email = ?",
124             (email,)
125         ).fetchone()
126
127         if not user:
128             flash("Incorrect credentials.")
129             return render_template("login.html")
130
131         if user["locked"]:
132             flash("Account is locked.")
133             return render_template("login.html")
134
135         if not check_password_hash(user["password_hash"], password):
136             db.execute(
137                 "UPDATE users SET failed_attempts = failed_attempts + 1 WHERE id = ?",
138                 (user["id"],)
139             )
140             if user["failed_attempts"] + 1 == 5:
141                 db.execute(
142                     "UPDATE users SET locked = 1 WHERE id = ?",
143                     (user["id"],)
144                 )
145                 flash("Account is locked after too many failed attempts.")
146             else:
147                 flash("Incorrect credentials.")
148             db.commit()
149             return render_template("login.html")
150
151         db.execute(
152             "UPDATE users SET failed_attempts = 0 WHERE id = ?",
153             (user["id"],)
154         )
155         db.commit()
156
157         # Redirect to the home page
158         return redirect(url_for("index"))
159
160 # The following code is for the home page
161 # It shows a list of all users in the database
162 # and a form to add a new user
163
164 @app.route("/")
165 def index():
166     db = get_db()
167     users = db.execute("SELECT * FROM users").fetchall()
168     return render_template("index.html", users=users)
169
170 @app.route("/add_user", methods=['POST'])
171 def add_user():
172     email = request.form.get("email")
173     password = request.form.get("password")
174     db = get_db()
175     db.execute(
176         "INSERT INTO users (email, password_hash) VALUES (?, ?)",
177         (email, generate_password_hash(password))
178     )
179     db.commit()
180     flash("User added successfully.")
181     return redirect(url_for("index"))
182
183 if __name__ == "__main__":
184     app.run(debug=True)
185 
```

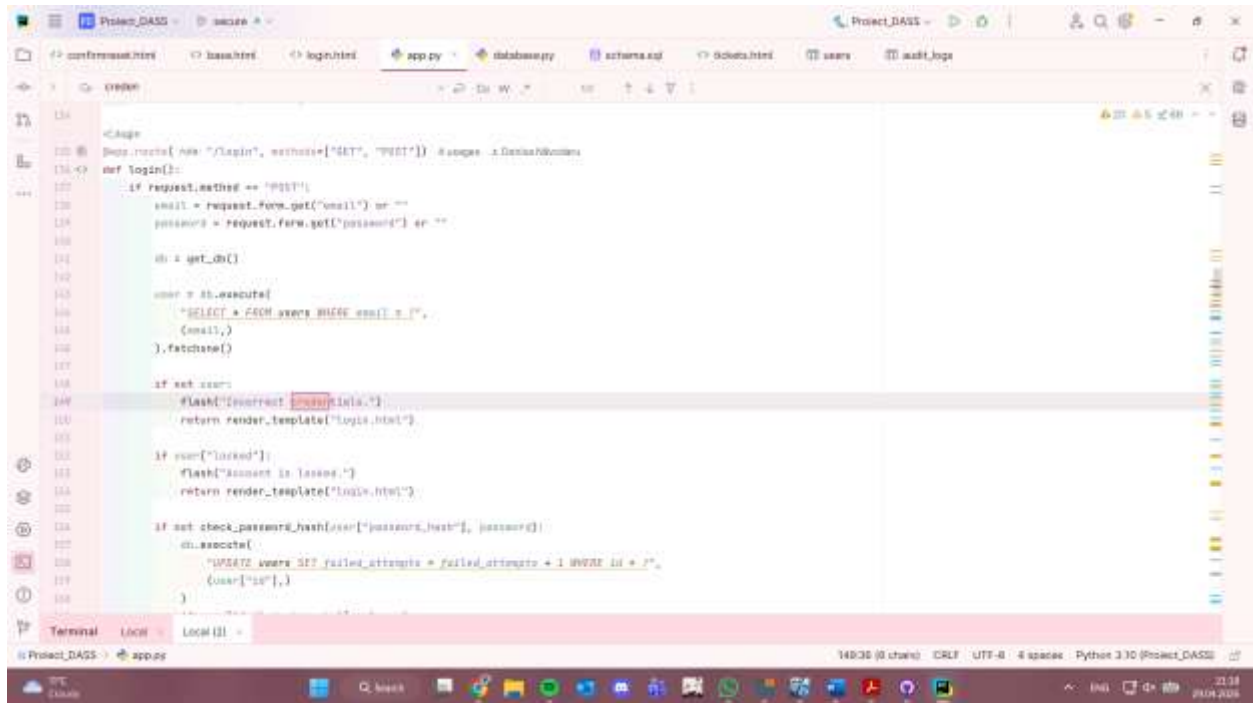


```

114 def login():
115     return render_template("login.html")
116
117 if not check_password_hash(user["password_hash"], password):
118     db.execute(
119         "UPDATE users SET failed_attempts = failed_attempts + 1 WHERE id = ?",
120         (user["id"],)
121     )
122     if user["failed_attempts"] + 1 == 5:
123         db.execute(
124             "UPDATE users SET locked = 1 WHERE id = ?",
125             (user["id"],)
126         )
127         flash("Account is locked after too many failed attempts.")
128     else:
129         flash("Incorrect credentials.")
130     db.commit()
131     return render_template("login.html")
132
133 db.execute(
134     "UPDATE users SET failed_attempts = 0 WHERE id = ?",
135     (user["id"],)
136 )
137 db.commit()
138
139 # Redirect to the home page
140 return redirect(url_for("index"))
141
142 # The following code is for the home page
143 # It shows a list of all users in the database
144 # and a form to add a new user
145
146 @app.route("/")
147 def index():
148     db = get_db()
149     users = db.execute("SELECT * FROM users").fetchall()
150     return render_template("index.html", users=users)
151
152 @app.route("/add_user", methods=['POST'])
153 def add_user():
154     email = request.form.get("email")
155     password = request.form.get("password")
156     db = get_db()
157     db.execute(
158         "INSERT INTO users (email, password_hash) VALUES (?, ?)",
159         (email, generate_password_hash(password))
160     )
161     db.commit()
162     flash("User added successfully.")
163     return redirect(url_for("index"))
164
165 if __name__ == "__main__":
166     app.run(debug=True)
167 
```

Dacă totuși reușește să își introducă parola înainte de al cincilea eșec, contorul de eșecuri va fi readus la 0.

7.4. User enumeration



```

134
135 <code>def login():
136     if request.method == 'POST':
137         email = request.form.get('email') or ''
138         password = request.form.get('password') or ''
139
140         db = get_db()
141
142         user = db.execute(
143             "SELECT * FROM users WHERE email = ?",
144             (email,)
145         ).fetchone()
146
147         if not user:
148             flash('Incorrect username.')
149             return render_template("login.html")
150
151         if user['locked']:
152             flash('Account is locked.')
153             return render_template("login.html")
154
155         if not check_password_hash(user['password_hash'], password):
156             db.execute(
157                 "UPDATE users SET failed_attempts = failed_attempts + 1 WHERE id = ?",
158                 (user['id'],)
159             )
160
161     else:
162         flash('Incorrect password.')
163         db.execute(
164             "UPDATE users SET failed_attempts = failed_attempts + 1 WHERE id = ?",
165             (user['id'],)
166         )
167
168     flash('Account is locked after too many failed attempts.')
169     flash('Incorrect username.')
170     db.commit()
171     return render_template("login.html")
172
173 db.execute(
174     "UPDATE users SET failed_attempts = 0 WHERE id = ?",
175     (user['id'],)
176 )
177 db.commit()
178
179 # in contextul acesta in user real se creaza sa user sa la in user sa la file se creaza user
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
1001
1002
1003
1004
1005
1006
1007
1008
1009
1010
1011
1012
1013
1014
1015
1016
1017
1018
1019
1020
1021
1022
1023
1024
1025
1026
1027
1028
1029
1030
1031
1032
1033
1034
1035
1036
1037
1038
1039
1040
1041
1042
1043
1044
1045
1046
1047
1048
1049
1050
1051
1052
1053
1054
1055
1056
1057
1058
1059
1060
1061
1062
1063
1064
1065
1066
1067
1068
1069
1070
1071
1072
1073
1074
1075
1076
1077
1078
1079
1080
1081
1082
1083
1084
1085
1086
1087
1088
1089
1090
1091
1092
1093
1094
1095
1096
1097
1098
1099
1100
1101
1102
1103
1104
1105
1106
1107
1108
1109
1110
1111
1112
1113
1114
1115
1116
1117
1118
1119
1120
1121
1122
1123
1124
1125
1126
1127
1128
1129
1130
1131
1132
1133
1134
1135
1136
1137
1138
1139
1140
1141
1142
1143
1144
1145
1146
1147
1148
1149
1150
1151
1152
1153
1154
1155
1156
1157
1158
1159
1160
1161
1162
1163
1164
1165
1166
1167
1168
1169
1170
1171
1172
1173
1174
1175
1176
1177
1178
1179
1180
1181
1182
1183
1184
1185
1186
1187
1188
1189
1190
1191
1192
1193
1194
1195
1196
1197
1198
1199
1200
1201
1202
1203
1204
1205
1206
1207
1208
1209
1210
1211
1212
1213
1214
1215
1216
1217
1218
1219
1220
1221
1222
1223
1224
1225
1226
1227
1228
1229
1230
1231
1232
1233
1234
1235
1236
1237
1238
1239
1240
1241
1242
1243
1244
1245
1246
1247
1248
1249
1250
1251
1252
1253
1254
1255
1256
1257
1258
1259
1260
1261
1262
1263
1264
1265
1266
1267
1268
1269
1270
1271
1272
1273
1274
1275
1276
1277
1278
1279
1280
1281
1282
1283
1284
1285
1286
1287
1288
1289
1290
1291
1292
1293
1294
1295
1296
1297
1298
1299
1300
1301
1302
1303
1304
1305
1306
1307
1308
1309
1310
1311
1312
1313
1314
1315
1316
1317
1318
1319
1320
1321
1322
1323
1324
1325
1326
1327
1328
1329
1330
1331
1332
1333
1334
1335
1336
1337
1338
1339
1340
1341
1342
1343
1344
1345
1346
1347
1348
1349
1350
1351
1352
1353
1354
1355
1356
1357
1358
1359
1360
1361
1362
1363
1364
1365
1366
1367
1368
1369
1370
1371
1372
1373
1374
1375
1376
1377
1378
1379
1380
1381
1382
1383
1384
1385
1386
1387
1388
1389
1390
1391
1392
1393
1394
1395
1396
1397
1398
1399
1400
1401
1402
1403
1404
1405
1406
1407
1408
1409
1410
1411
1412
1413
1414
1415
1416
1417
1418
1419
1420
1421
1422
1423
1424
1425
1426
1427
1428
1429
1430
1431
1432
1433
1434
1435
1436
1437
1438
1439
1440
1441
1442
1443
1444
1445
1446
1447
1448
1449
1450
1451
1452
1453
1454
1455
1456
1457
1458
1459
1460
1461
1462
1463
1464
1465
1466
1467
1468
1469
1470
1471
1472
1473
1474
1475
1476
1477
1478
1479
1480
1481
1482
1483
1484
1485
1486
1487
1488
1489
1490
1491
1492
1493
1494
1495
1496
1497
1498
1499
1500
1501
1502
1503
1504
1505
1506
1507
1508
1509
1510
1511
1512
1513
1514
1515
1516
1517
1518
1519
1520
1521
1522
1523
1524
1525
1526
1527
1528
1529
1530
1531
1532
1533
1534
1535
1536
1537
1538
1539
1540
1541
1542
1543
1544
1545
1546
1547
1548
1549
1550
1551
1552
1553
1554
1555
1556
1557
1558
1559
1560
1561
1562
1563
1564
1565
1566
1567
1568
1569
1570
1571
1572
1573
1574
1575
1576
1577
1578
1579
1580
1581
1582
1583
1584
1585
1586
1587
1588
1589
1590
1591
1592
1593
1594
1595
1596
1597
1598
1599
1600
1601
1602
1603
1604
1605
1606
1607
1608
1609
1610
1611
1612
1613
1614
1615
1616
1617
1618
1619
1620
1621
1622
1623
1624
1625
1626
1627
1628
1629
1630
1631
1632
1633
1634
1635
1636
1637
1638
1639
1640
1641
1642
1643
1644
1645
1646
1647
1648
1649
1650
1651
1652
1653
1654
1655
1656
1657
1658
1659
1660
1661
1662
1663
1664
1665
1666
1667
1668
1669
1670
1671
1672
1673
1674
1675
1676
1677
1678
1679
1680
1681
1682
1683
1684
1685
1686
1687
1688
1689
1690
1691
1692
1693
1694
1695
1696
1697
1698
1699
1700
1701
1702
1703
1704
1705
1706
1707
1708
1709
1710
1711
1712
1713
1714
1715
1716
1717
1718
1719
1720
1721
1722
1723
1724
1725
1726
1727
1728
1729
1730
1731
1732
1733
1734
1735
1736
1737
1738
1739
1740
1741
1742
1743
1744
1745
1746
1747
1748
1749
1750
1751
1752
1753
1754
1755
1756
1757
1758
1759
1760
1761
1762
1763
1764
1765
1766
1767
1768
1769
1770
1771
1772
1773
1774
1775
1776
1777
1778
1779
1780
1781
1782
1783
1784
1785
1786
1787
1788
1789
1790
1791
1792
1793
1794
1795
1796
1797
1798
1799
1800
1801
1802
1803
1804
1805
1806
1807
1808
1809
1810
1811
1812
1813
1814
1815
1816
1817
1818
1819
1820
1821
1822
1823
1824
1825
1826
1827
1828
1829
1830
1831
1832
1833
1834
1835
1836
1837
1838
1839
1840
1841
1842
1843
1844
1845
1846
1847
1848
1849
1850
1851
1852
1853
1854
1855
1856
1857
1858
1859
1860
1861
1862
1863
1864
1865
1866
1867
1868
1869
1870
1871
1872
1873
1874
1875
1876
1877
1878
1879
1880
1881
1882
1883
1884
1885
1886
1887
1888
1889
1890
1891
1892
1893
1894
1895
1896
1897
1898
1899
1900
1901
1902
1903
1904
1905
1906
1907
1908
1909
1910
1911
1912
1913
1914
1915
1916
1917
1918
1919
1920
1921
1922
1923
1924
1925
1926
1927
1928
1929
1930
1931
1932
1933
1934
1935
1936
1937
1938
1939
1940
1941
1942
1943
1944
1945
1946
1947
1948
1949
1950
1951
1952
1953
1954
1955
1956
1957
1958
1959
1960
1961
1962
1963
1964
1965
1966
1967
1968
1969
1970
1971
1972
1973
1974
1975
1976
1977
1978
1979
1980
1981
1982
1983
1984
1985
1986
1987
1988
1989
1990
1991
1992
1993
1994
1995
1996
1997
1998
1999
2000
2001
2002
2003
2004
2005
2006
2007
2008
2009
2010
2011
2012
2013
2014
2015
2016
2017
2018
2019
2020
2021
2022
2023
2024
2025
2026
2027
2028
2029
2030
2031
2032
2033
2034
2035
2036
2037
2038
2039
2040
2041
2042
2043
2044
2045
2046
2047
2048
2049
2050
2051
2052
2053
2054
2055
2056
2057
2058
2059
2060
2061
2062
2063
2064
2065
2066
2067
2068
2069
2070
2071
2072
2073
2074
2075
2076
2077
2078
2079
2080
2081
2082
2083
2084
2085
2086
2087
2088
2089
2090
2091
2092
2093
2094
2095
2096
2097
2098
2099
2100
2101
2102
2103
2104
2105
2106
2107
2108
2109
2110
2111
2112
2113
2114
2115
2116
2117
2118
2119
2120
2121
2122
2123
2124
2125
2126
2127
2128
2129
2130
2131
2132
2133
2134
2135
2136
2137
2138
2139
2140
2141
2142
2143
2144
2145
2146
2147
2148
2149
2150
2151
2152
2153
2154
2155
2156
2157
2158
2159
2160
2161
2162
2163
2164
2165
2166
2167
2168
2169
2170
2171
2172
2173
2174
2175
2176
2177
2178
2179
2180
2181
2182
2183
2184
2185
2186
2187
2188
2189
2190
2191
2192
2193
2194
2195
2196
2197
2198
2199
2200
2201
2202
2203
2204
2205
2206
2207
2208
2209
2210
2211
2212
2213
2214
2215
2216
2217
2218
2219
2220
2221
2222
2223
2224
2225
2226
2227
2228
2229
2230
2231
2232
2233
2234
2235
2236
2237
2238
2239
2240
2241
2242
2243
2244
2245
2246
2247
2248
2249
2250
2251
2252
2253
2254
2255
2256
2257
2258
2259
2260
2261
2262
2263
2264
2265
2266
2267
2268
2269
2270
2271
2272
2273
2274
2275
2276
2277
2278
2279
2280
2281
2282
2283
2284
2285
2286
2287
2288
2289
2290
2291
2292
2293
2294
2295
2296
2297
2298
2299
2300
2301
2302
2303
2304
2305
2306
2307
2308
2309
2310
2311
2312
2313
2314
2315
2316
2317
2318
2319
2320
2321
2322
2323
2324
2325
2326
2327
2328
2329
2330
2331
2332
2333
2334
2335
2336
2337
2338
2339
2340
2341
2342
2343
2344
2345
2346
2347
2348
2349
2350
2351
2352
2353
2354
2355
2356
2357
2358
2359
2360
2361
2362
2363
2364
2365
2366
2367
2368
2369
2370
2371
2372
2373
2374
2375
2376
2377
2378
2379
2380
2381
2382
2383
2384
2385
2386
2387
2388
2389
2390
2391
2392
2393
2394
2395
2396
2397
2398
2399
2400
2401
2402
2403
2404
2405
2406
2407
2408
2409
2410
2411
2412
2413
2414
2415
2416
2417
2418
2419
2420
2421
2422
2423
2424
2425
2426
2427
2428
2429
2430
2431
2432
2433
2434
2435
2436
2437
2438
2439
2440
2441
2442
2443
2444
2445
2446
2447
2448
2449
2450
2451
2452
2453
2454
2455
2456
2457
2458
2459
2460
2461
2462
2463
2464
2465
2466
2467
2468
2469
2470
2471
2472
2473
2474
2475
2476
2477
2478
2479
2480
2481
2482
2483
2484
2485
2486
2487
2488
2489
2490
2491
2492
2493
2494
2495
2496
2497
2498
2499
2500
2501
2502
2503
2504
2505
2506
2507
2508
2509
2510
2511
2512
2513
2514
2515
2516
2517
2518
2519
2520
2521
2522
2523
2524
2525
2526
2527
2528
2529
2530
2531
2532
2533
2534
2535
2536
2537
2538
2539
2540
2541
2542
2543
2544
2545
2546
2547
2548
2549
2550
2551
2552
2553
2554
2555
2556
2557
2558
2559
2560
2561
2562
2563
2564
2565
2566
2567
2568
2569
2570
2571
2572
2573
2574
2575
2576
2577
2578
2579
2580
2581
2582
2583
2584
2585
2586
2587
2588
2589
2590
2591
2592
2593
2594
2595
2596
2597
2598
2599
2600
2601
2602
2603
2604
2605
2606
2607
2608
2609
2610
2611
2612
2613
2614
2615
2616
2617
2618
2619
2620
2621
2622
2623
2624
2625
2626
2627
2628
2629
2630
2631
2632
2633
2634
2635
2636
2637
2638
2639
2640
2641
2642
2643
2644
2645
2646
2647
2648
2649
2650
2651
2652
2653
2654
2655
2656
2657
2658
2659
2660
2661
2662
2663
2664
2665
2666
2667
2668
2669
2670
2671
2672
2673
2674
2675
2676
2677
2678
2679
2680
2681
2682
2683
```

```

1 import re
2 from flask import Flask, request, jsonify
3
4 app = Flask(__name__)
5 csrf = CSRFProtect(app)
6 app.config['SECRET_KEY'] = secrets.token_hex(16)
7 app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///data.db'
8
9 session_cookie_name = 'session',
10 session_cookie_secure = False,
11 session_cookie_path = '/',
12 session_cookie_max_age = 3600,
13 session_cookie_http_only = True,
14 session_cookie_samesite = 'lax'
15
16 permanent_session_lifetime = timedelta(minutes=10)
17
18 @app.cli.command()
19 def init_db():
20     init_db()
21     log_action('user:1', action="initialized database", resource="database", resource_id=1)
22     print("initialized the database.")
23
24 @app.cli.command()
25 def migrate_db():
26     db = get_db()
27     try:
28         db.execute("ALTER TABLE users ADD COLUMN failed_attempts INTEGER NOT NULL DEFAULT 0")
29     except Exception as e:
30         print(f"failed attempt: {e}")
31     try:
32         db.execute("ALTER TABLE users ADD COLUMN reset_token STRING")
33     except Exception as e:
34         print(f"failed attempt: {e}")
35
36 if __name__ == '__main__':
37     app.run(debug=True)

```

[illegible]


```
def login_required():
    return redirect(url_for("login"))

@login_required
def profile():
    user = current_user()
    return render_template("profile.html", user=user)

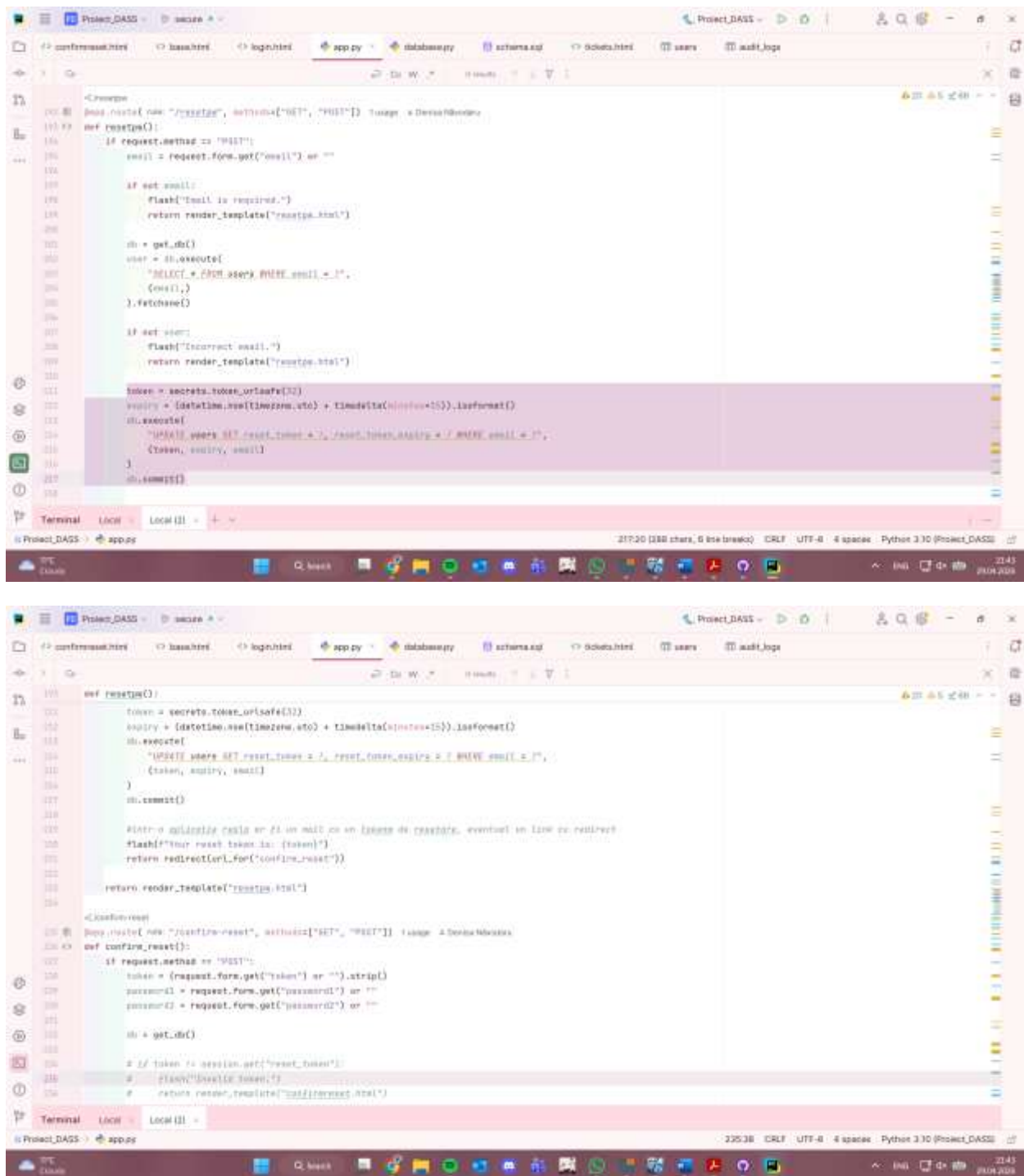
def add_security_headers(response):
    response.headers["X-Content-Type-Options"] = "nosniff"
    response.headers["X-Frame-Options"] = "DENY"
    response.headers["X-XSS-Protection"] = "1; mode=block"
    response.headers["Content-Security-Policy"] = "default-src 'self'; script-src 'self' 'unsafe-inline'"
    return response

if __name__ == "__main__":
    app.run(host = "0.0.0.0", port = 1099)
```

[illegible]

Pentru a gestiona mai sigur sesiunile, am integrat cookies cu token CSRF. Fiecare formular primește un token unic generat pe server, care dacă lipsește sau e invalid la submit va rezulta în eroare de server, deci cererea va fi respinsă. CSP previne și el script-uri din afara self-ului din a fi rulate (motiv pentru care am avut eroare cu scriptul de show password). Headerele de la final previn câteva atacuri mai întâlnite, cum ar fi clickhijacking. În plus, după 10 minute sesiunea unui utilizator va deveni invalidă și va fi nevoie de un relogin.

7.6. Resetare parolă nesigură



The image displays two screenshots of a code editor, likely VS Code, showing Python code for a password reset functionality in a Flask application. The code is organized into two functions: `resetpw()` and `confirm_reset()`.

First Screenshot (Top): Shows the `resetpw()` function. It handles a POST request to `/resetpw`. It checks if the request method is 'POST'. If not, it returns `render_template("resetpw.html")`. If it is 'POST', it retrieves the email from the request form. It then checks if the email is required. If not, it flashes an error message and returns the template. If the email is present, it queries the database for a user with that email. If no user is found, it flashes an error message and returns the template. If a user is found, it generates a reset token and an expiry time, updates the user's reset token and expiry in the database, and returns the template.

Second Screenshot (Bottom): Shows the `confirm_reset()` function. It handles a POST request to `/confirm-reset`. It checks if the request method is 'POST'. If not, it returns `render_template("confirm_reset.html")`. If it is 'POST', it retrieves the token, password1, and password2 from the request form. It then checks if the token is required. If not, it flashes an error message and returns the template. If the token is present, it queries the database for a user with that token. If no user is found, it flashes an error message and returns the template. If a user is found, it checks if the passwords match. If they do, it updates the user's password and returns the template. If they do not, it flashes an error message and returns the template.

```

<!--resetpw.html-->
<form method="POST">
    {{ form.csrf_token }}
    {{ form.email }}
    {{ form.submit }}
</form>

def resetpw():
    if request.method == 'POST':
        email = request.form.get("email") or ""

        if not email:
            flash("Email is required.")
            return render_template("resetpw.html")

        db = get_db()
        user = db.execute(
            'SELECT * FROM users WHERE email = ?',
            (email,)
        ).fetchone()

        if not user:
            flash("Incorrect email.")
            return render_template("resetpw.html")

        token = secrets.token_urlsafe(32)
        expiry = (datetime.now(timedelta.utc) + timedelta(minutes=15)).isoformat()
        db.execute(
            'UPDATE users SET reset_token = ?, reset_token_expiry = ? WHERE email = ?',
            (token, expiry, email)
        )
        db.commit()

        flash("Your reset token is: {}".format(token))
        return redirect(url_for("confirm_reset"))

    return render_template("resetpw.html")

<!--confirm_reset.html-->
<form method="POST">
    {{ form.csrf_token }}
    {{ form.token }}
    {{ form.password1 }}
    {{ form.password2 }}
    {{ form.submit }}
</form>

def confirm_reset():
    if request.method == 'POST':
        token = request.form.get("token") or ""
        password1 = request.form.get("password1") or ""
        password2 = request.form.get("password2") or ""

        db = get_db()

        # if token != session.get("reset_token"):
        #     flash("Invalid token.")
        #     return render_template("confirm_reset.html")

        user = db.execute(
            'SELECT * FROM users WHERE reset_token = ?',
            (token,)
        ).fetchone()

        if not user:
            flash("Invalid token.")
            return render_template("confirm_reset.html")

        if password1 != password2:
            flash("Passwords do not match.")
            return render_template("confirm_reset.html")

        db.execute(
            'UPDATE users SET password = ? WHERE reset_token = ? AND reset_token_expiry >= ?',
            (password1, token, datetime.now(timedelta.utc))
        )
        db.commit()

        flash("Password reset successful.")
        return redirect(url_for("login"))

    return render_template("confirm_reset.html")
    
```



```

def confirm_reset():
    # if token != session.get("reset_token"):
    #     flash("Invalid token.")
    #     return render_template("confirm_reset.html")

    if password1 != password2:
        flash("Passwords do not match.")
        return render_template("confirm_reset.html")

    user = db.session.query(User).filter_by(id=session.get("user_id")).first()

    if not user:
        flash("Invalid token.")
        return render_template("confirm_reset.html")

    expiry = datetime.datetime.utcnow() + timedelta(minutes=15)
    if expiry < datetime.datetime.utcnow():
        flash("Token expired.")
        return render_template("confirm_reset.html")

    email = user.email
    username = user.username

    db = get_db()
    user = db.session.query(User).filter_by(email=email).first()

    # if user:
    #     flash("Email already exists.")
    #     return render_template("confirm_reset.html")

    user.password = password1
    user.reset_token = session.get("reset_token")
    user.reset_token_expiry = expiry
    db.session.commit()

    flash("Password successfully changed.")
    return redirect(url_for("login"))

    return render_template("confirm_reset.html")
    
```

```

def confirm_reset():
    if not session.get("password_reset_token"):
        flash("Password reset token is missing.")
        return render_template("confirm_reset.html")

    token = session.get("password_reset_token")

    user = db.session.query(User).filter_by(reset_token=token).first()

    if not user:
        flash("Invalid token.")
        return render_template("confirm_reset.html")

    expiry = datetime.datetime.utcnow() + timedelta(minutes=15)
    if expiry < datetime.datetime.utcnow():
        flash("Token expired.")
        return render_template("confirm_reset.html")

    email = user.email
    username = user.username

    db = get_db()
    user = db.session.query(User).filter_by(email=email).first()

    # if user:
    #     flash("Email already exists.")
    #     return render_template("confirm_reset.html")

    user.password = password1
    user.reset_token = session.get("reset_token")
    user.reset_token_expiry = expiry
    db.session.commit()

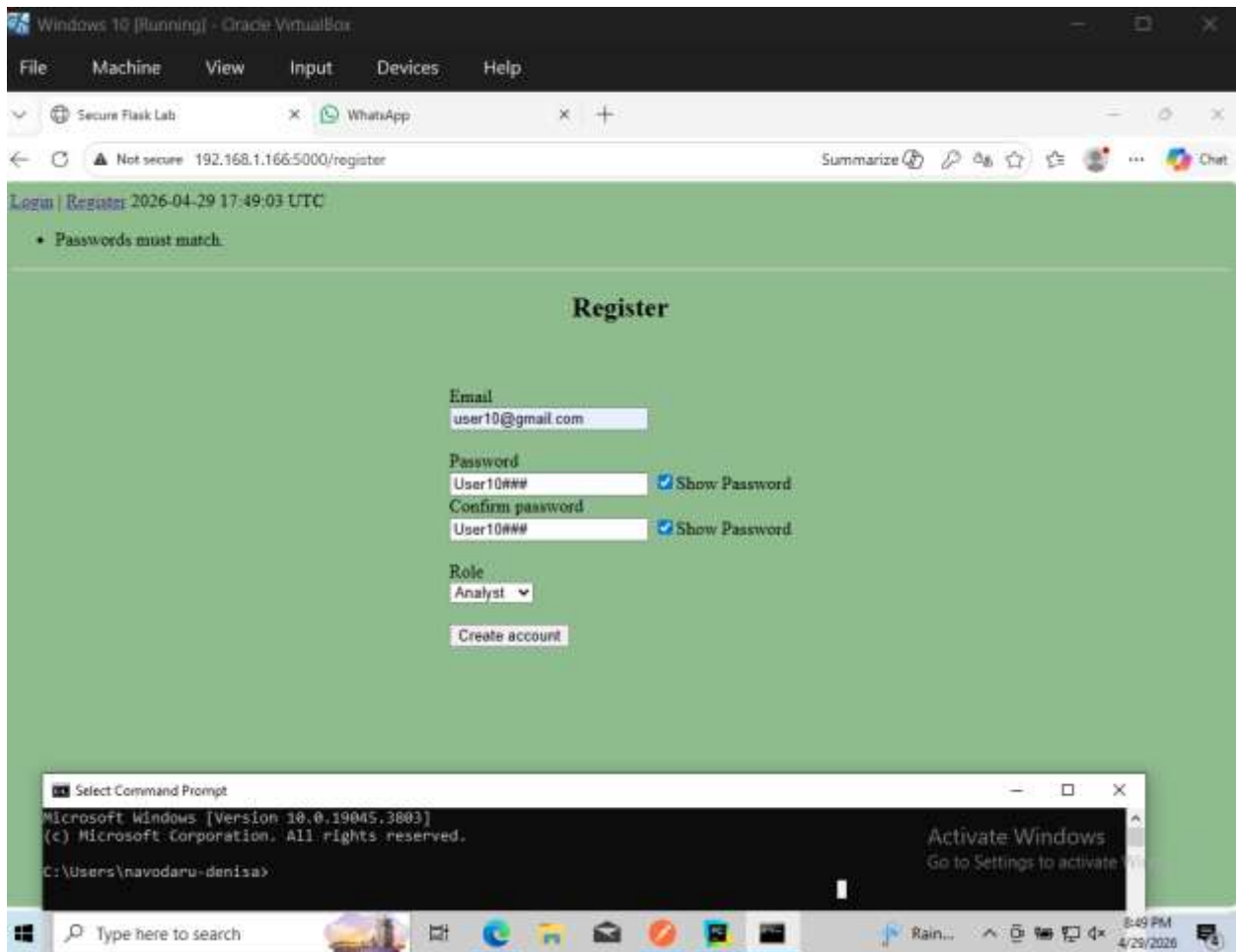
    flash("Password successfully changed.")
    return redirect(url_for("login"))

    return render_template("confirm_reset.html")
    
```

Spre deosebire de varianta vulnerabilă unde token-ul era static (1234 mereu), în noua versiune este generat prin biblioteca secrets pentru a nu fi ghicit de atacatori. În plus, acesta expiră după 15 minute, deci și pentru un script foarte puternic ar fi dificil ca în doar 15 minute să dea de cap unui token generat astfel.

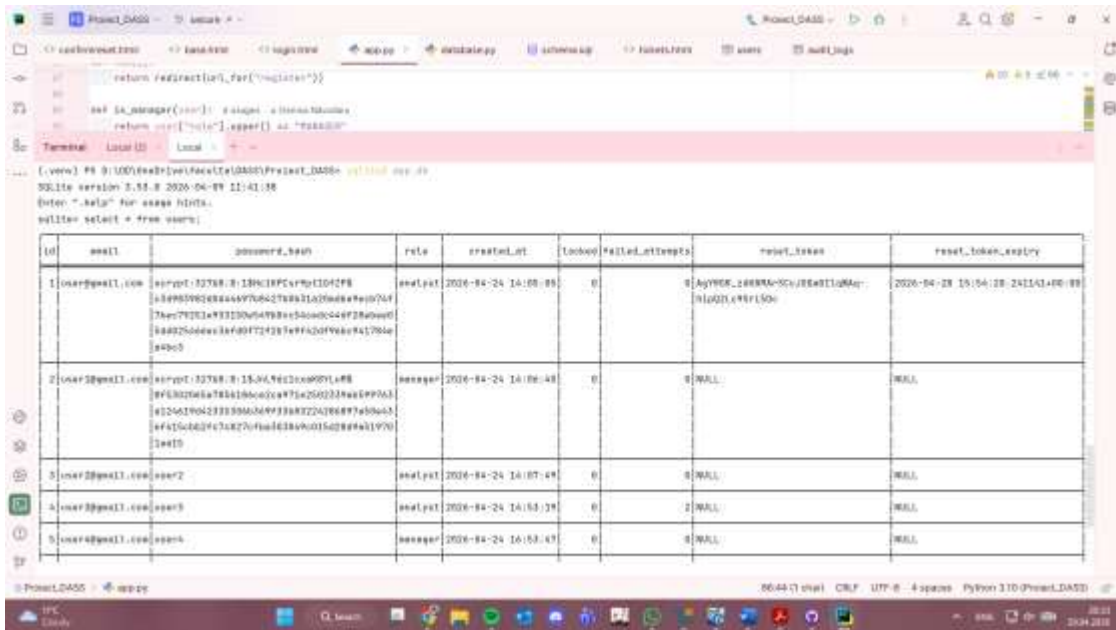
8. Re-test

8.1. Password policy slab



Cu o parolă ca aceasta, script-ul meu simplu va eșua deoarece nu verific cu parole așa complexe. Desigur, un atacator poate face asta, însă ar fi nevoie de multe resurse pentru a face proba cu un număr așa de mare de parole.

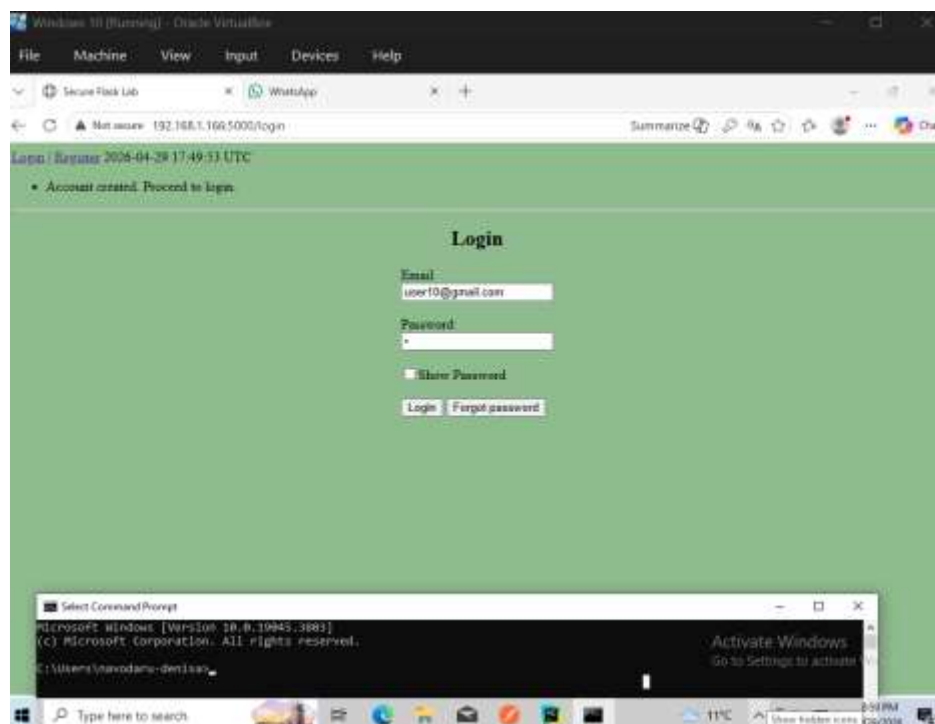
8.2. Stocare nesigură a parolelor

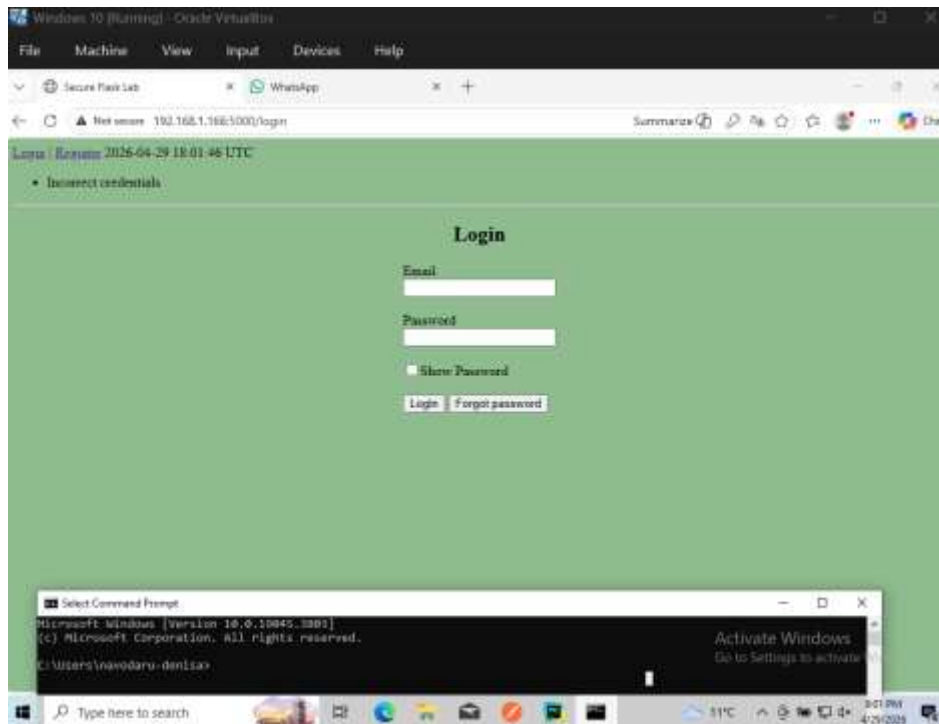


Cum am menționat și la 6.2, am parolele stocate în ambele feluri din cauza switch-ului între versiuni și adăugarea de utilizatori și schimbarea de parole. Cele criptate sunt cele modificate pe versiunea securizată.

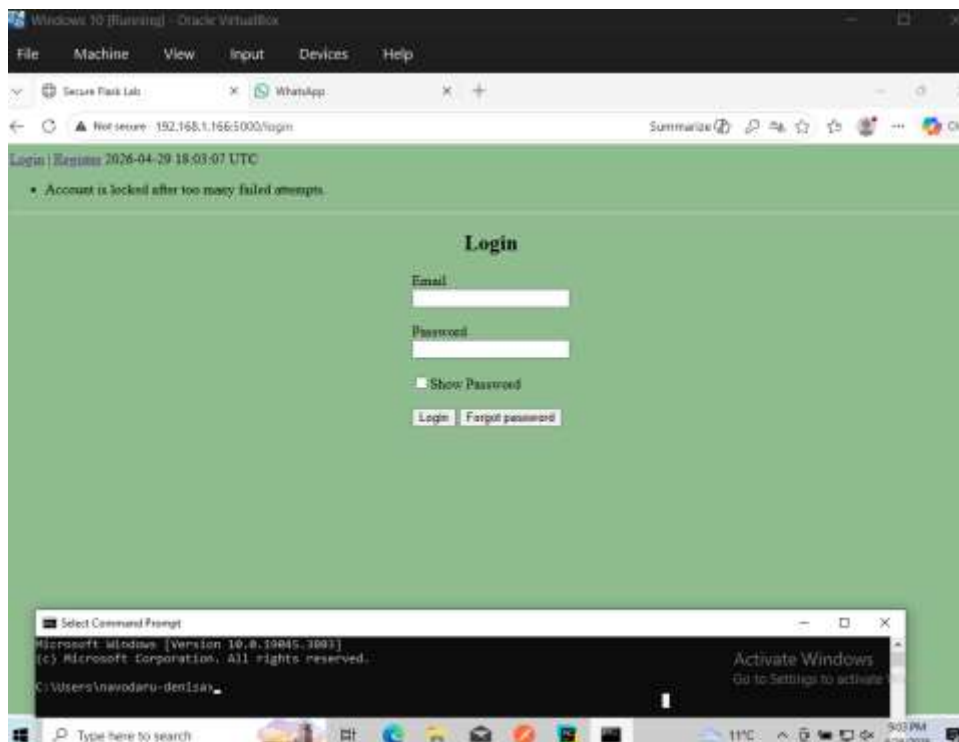
8.3. Brute force / lipsă rate limiting

Voi încerca să mă loghez în contul creat pentru punctul 8.1 cu o parolă greșită.





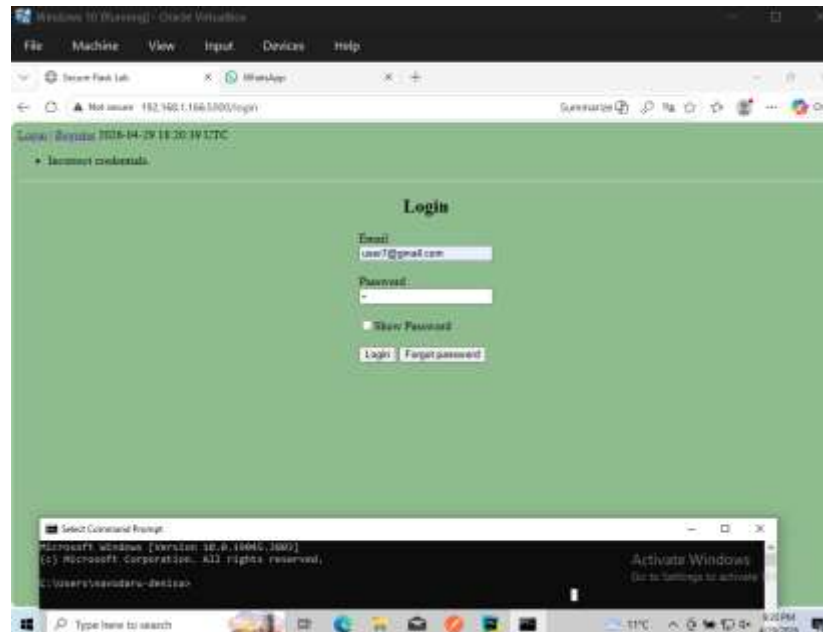
Voi repeta aceeași acțiune de încă 4 ori. După a cincea încercare voi vedea:



Astfel, un script care rulează parole generate automat, sau un atacator foarte dedicat, va avea un număr foarte limitat de încercări pentru a ghici parola, după care contul va fi blocat. Dacă

aplicația ar fi reală, utilizatorul ar trebui să sune la Customer Support pentru a primi un token de resetare a parolei și să își poată reaccesa contul.

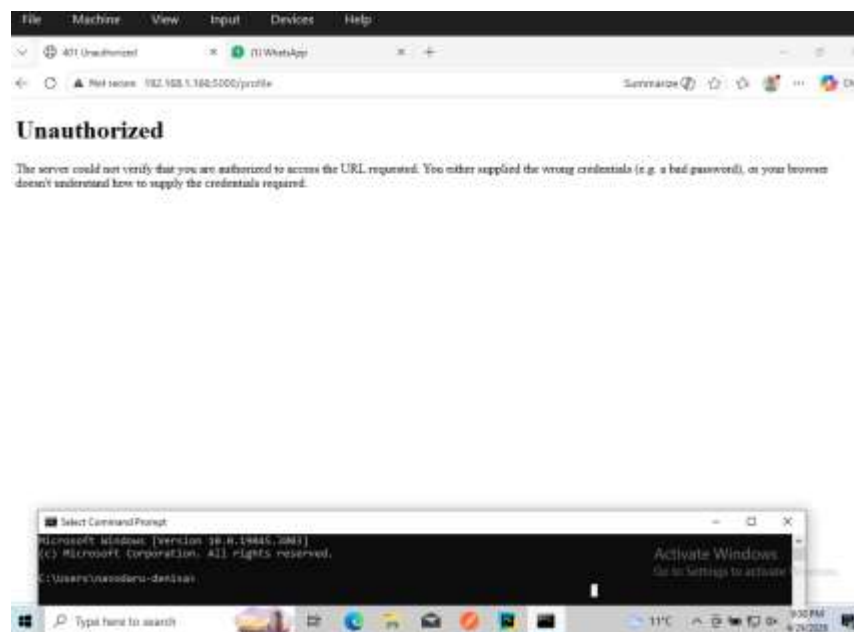
8.4. User enumeration



În consecință, un atacator nu va mai putea face rost de o listă a utilizatorilor prin încercări nenumărate de login cu email-uri diferite.

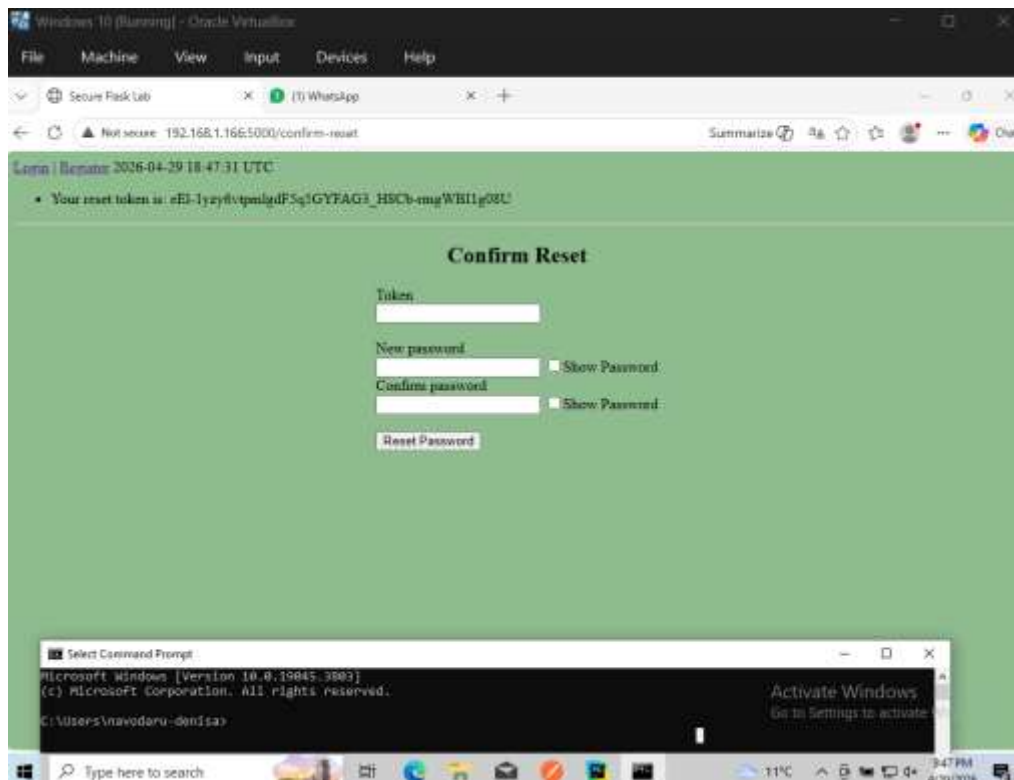
8.5. Gestionare nesigura a sesiunilor

Mesaj afișat după ce se încearcă navigarea aplicației după expirarea sesiunii:



The screenshot shows a Windows 10 desktop environment. The top taskbar displays the Start button, a search bar, and several pinned application icons. The main window is a web browser with multiple tabs. The active tab shows a user profile page for 'h4llie' with the email 'h4llie@gmail.com' and the role 'analyst'. The page includes a 'Profile' section with the email and role, and a 'Tickets' link. A sidebar on the left lists various application categories like Manifest, Service work..., Storage, Storage..., Background serv..., Background t..., Background..., and Source track... The right sidebar shows a 'Cookie Value' table with columns for Name, Value, Domain, Path, Size, HTTP, Secure, SameSite, and Expires. The table contains one entry for 'session'. Below the table, there is a 'Cookie Value' section with a 'Show URL-decoded' checkbox and a text area containing a long, encoded string. In the bottom-left corner, a 'Select Command Prompt' window is open, showing the Windows version (1809) and the user's home directory (C:\Users\navedaru-denis...). The bottom-right corner of the desktop features an 'Activate Windows' watermark and a system tray with the date and time (9:41 PM, 4/26/2018).

8.6. Resetare parolă nesigură



9. Audit & logging

Pentru diferite acțiuni am log actions care îmi intră în audit_logs. Acestea sunt: login, logout, reset password, create ticket, edit ticket, new user. La fiecare acțiune există un timestamp (setat pe UTC), numele acțiunii și id-ul celui ce a acționat, cât și id-ul resursei afectate și ip-ul de pe care a fost accesată pagina la momentul acțiunii. Un utilizator cu rol de manager are acces la tabelul de audit printr-un link de pe profilul său.

The top screenshot shows the 'Profile' page of a user named 'Eduard Sorin'. It includes a 'Logout' button and a link to 'Audit Logs'. The bottom screenshot shows the 'Audit Logs' page, which displays a table of logged actions.

ID	Action	Resource ID	Timestamp	IP
1	new user added	1	2024-04-23 10:00:00	127.0.0.1
2	new user added	1	2024-04-23 10:00:00	127.0.0.1
3	new user added	1	2024-04-23 10:00:00	127.0.0.1
4	new user added	1	2024-04-23 10:00:00	127.0.0.1
5	new user added	1	2024-04-23 10:00:00	127.0.0.1
6	new user added	1	2024-04-23 10:00:00	127.0.0.1
7	new user added	1	2024-04-23 10:00:00	127.0.0.1
8	new user added	1	2024-04-23 10:00:00	127.0.0.1
9	new user added	1	2024-04-23 10:00:00	127.0.0.1
10	new user added	1	2024-04-23 10:00:00	127.0.0.1
11	new user added	1	2024-04-23 10:00:00	127.0.0.1
12	new user added	1	2024-04-23 10:00:00	127.0.0.1
13	new user added	1	2024-04-23 10:00:00	127.0.0.1
14	new user added	1	2024-04-23 10:00:00	127.0.0.1

M-am jucat puțin cu numele de la acțiuni la început, deci câteva dintre ele diferă în primele intrări.

Baza de date inițializată:

```

19 @app.cli.command("init-db")  # Denisa Năvodaru
20 def init_db_command():
21     init_db()
22     log_action( user_id: 0, action: "initialized database", resource: "database", resource_id: 1)
23     print("Initialized the database.")

```

Utilizator nou înregistrat:

```

try:
    cursor = db.execute(
        "INSERT INTO users (email, password_hash, role) VALUES (?, ?, ?)",
        (email, pw_hash, role),
    )
    userid = cursor.lastrowid
    db.commit()
    log_action(userid, action: "new user added", resource: "user", userid)

```

Login:

```

183 session.clear()
184 session["user_id"] = user["id"]
185 session.permanent = True
186 flash(f"Logged in as: {user['email']}")
187 log_action(user["id"], action: "login", resource: "user", user["id"])
188 return redirect(url_for("profile"))

```

Utilizatorul a resetat parola contului:

```

283 db.commit()
284 log_action(userid, action: "reset pw", resource: "user", userid)
285

```

Utilizatorul a creat un tichet:

```

313 db.commit()
314
315 log_action(user["id"], action: "created ticket", resource: "ticket", ticket_id)
316 return redirect(url_for("list_tickets"))
317

```

Utilizatorul a editat un tichet:

```

398
399 db.commit()
400 log_action(user["id"], action: "edited ticket", resource: "ticket", id)
401

```

Logout:

```

418 @app.post("/logout") 1 usage  Denisa Năvodaru
419 def logout():
420     user = current_user()
421     if user:
422         log_action(user["id"], action: "logout", resource: "user", user["id"])
423     session.clear()
424     return redirect(url_for("login"))
425

```

Pentru timestamp și IP am valori setate default în funcția de log action.

```

29 def log_action(user_id, action, resource, resource_id): 8 usages  Denisa Năvodaru
30     db = get_db()
31     db.execute(
32         sql: """
33         INSERT INTO audit_logs (user_id, action, resource, resource_id, timestamp, ip_address)
34         VALUES (?, ?, ?, ?, ?, ?)
35         """,
36         parameters: (
37             user_id,
38             action,
39             resource,
40             str(resource_id),
41             datetime.now(timezone.utc),
42             request.remote_addr if request else "unknown"
43         )
44     )
45     db.commit()

```

10. Concluzii

Având posibilitatea de a ști potențialele vulnerabilități ale unei aplicații înainte de a o implementa, procesul de a construi o aplicație intenționat ușor de spart mi-a deschis ochii la greșelile pe care le-am făcut la proiecte anterioare. Deși unele lucruri sunt destul de intuitive, unele par așa de banale încât e ușor uneori să le trec cu vederea, mai ales pentru că nu o gândesc ca o aplicație pe care un atacator ar ținti-o. Să îmi las singură loc de greșeală pentru un ulterior fix a fost o metodă de învățare foarte constructivă pentru mine, fiind mai ușor de sesizat golurile înainte să apară, și nu abia la final.

Căutând vulnerabilitățile pe OWASP mi-am dat seama cât de multe trec cu vederea și câte porțițe există pentru un atacator. Mi-am dat seama că există aplicații pe care eu le folosesc ce sunt supuse unor vulnerabilități listate doar în proiectul acesta, deci sunt convinsă că au și altele pe care nu le-am abordat pe pielea mea.